

ÚJRAHASZNOSÍTOTT MOBILTELEFONOKON ALAPULÓ BIZTONSÁGI RENDSZER MEGVALÓSÍTÁSA

REALIZACIJA BEZBEDNOSNOG SISTEMA ZASNOVANOG NA RECIKLIRANIM MOBILNIM TELEFONIMA

IMPLEMENTATION OF A SECURITY SYSTEM BASED ON RECYCLED MOBILE PHONES

Hallgató

VASS RICHÁRD
26223071

Mentor

DR. PLETL SZILVESZTER

Szabadka,
2026

Hallgatói nyilatkozat

Kijelentem, hogy e szakdolgozat saját munkám eredménye. A szakdolgozat elkészítésekor felhasznált szakirodalomra és egyéb eszközökre maradéktalanul hivatkoztam. A Szabadkai Műszaki Szakfőiskola a szakdolgozatomban bemutatott eredményeimet, igényei szerint, szabadon és térítésmentesen felhasználhatja, szem előtt tartva az esetleges titkosításra vonatkozó megállapodást, amely ebben az esetben a szakdolgozat részét képezi.

Szabadkán,
kelt

Hallgató aláírása

Absztrakt

A dolgozat egy önálló fejlesztésű, felhőfüggetlen, lokális hálózaton működő biztonsági kamerarendszer, a SwarmCam tervezését és megvalósítását mutatja be, amelynek célja a háztartásokban feleslegessé vált Android okostelefonok IP kameraként történő újrahasznosítása. A telefonokon futó, nyílt forráskódú Android IP Camera alkalmazás HTTPS protokollon szolgáltatja a hardveresen kódolt H.264 videófolyamot és a készülék állapotadatait; ezeket egy Docker-alapú szerverkörnyezetben futó Frigate NVR dolgozza fel valós idejű, mesterséges intelligencia alapú objektumdetekcióval. A rendszer magját egy aszinkron Python (FastAPI) háttér szolgáltatás képezi, amely automatikus hálózati felderítéssel azonosítja az elérhető kamerákat, MQTT protokollon közvetíti a detektált eseményeket, WebSocket kapcsolaton át valós időben tájékoztatja a React alapú, asztali és mobil elrendezést egyaránt támogató kezelőfelületet, az élő videót pedig újrakódolás nélkül, fMP4 szegmensekként játssza át a böngészőnek. A rendszert egy különálló, YOLO-alapú mikroszolgáltatás egészíti ki tűz- és füstérzékeléssel. A dolgozat bemutatja a rendszer architektúráját, a kommunikációs rétegek (REST, WebSocket, SSE, MQTT) tervezési döntéseit, a konténerizált üzemeltetést, valamint a megvalósítás és a valós környezetben végzett, számszerű mérésekkel alátámasztott tesztelés tapasztalatait. Az eredmények igazolják, hogy elavult mobiltelefonokból, dedikált kamerahardver nélkül is kiépíthető egy megbízható, bővíthető és teljes mértékben a felhasználó tulajdonában maradó megfigyelőrendszer.

Kulcsszavak: Android, e-hulladék, elosztott rendszer, FastAPI, mesterséges intelligencia, objektumdetekció, MQTT, konténerizáció

Tartalom

Hallgatói nyilatkozat.....	2
Absztrakt.....	3
Tartalom.....	4
Jelmagyarázat.....	6
Köszönetnyilvánítás.....	8
A szakdolgozat témája.....	9
1. Bevezető.....	11
1.1. A probléma leírása.....	11
1.2. A szakdolgozat céljai.....	11
2. Szakirodalmi áttekintés.....	13
2.1. Az otthoni videómegfigyelés technológiai környezete.....	13
2.1.1. A felhőalapú konzumer rendszerek működési modellje.....	13
2.1.2. Az edge, azaz helyben futó NVR alternatívája.....	14
2.2. Kamerahardver: szabványos IP kamerák és fogyasztói kameraalkalmazások.....	15
2.2.1. Az ONVIF szabvány és az RTSP protokoll.....	15
2.2.2. A fogyasztói IP-kamera alkalmazások.....	16
2.3. Objektumdetekció a videóalapú megfigyelésben.....	17
2.3.1. A klasszikus mozgásdetekció és korlátai.....	17
2.3.2. Konvolúciós objektumdetektorok.....	17
2.3.3. A Frigate detekciós csővezetéke.....	18
2.4. Kommunikációs minták elosztott rendszerekben.....	18
2.5. Konténerizáció és szolgáltatásorientált üzemeltetés.....	19
2.6. Elektronikai hulladék és a mobileszközök újrahasznosítása.....	20
3. Problémamegoldás.....	22
3.1. Követelmények és tervezési kényszerek.....	22
3.2. A SwarmCam architektúrája.....	23
3.3. A hálózati és kommunikációs réteg tervezése.....	24

3.4.	Konténerizáció és üzemeltetés.....	26
3.5.	Implementáció.....	27
3.5.1.	Automatikus hálózati felderítés.....	27
3.5.2.	A kamera-protokoll cseréje és annak következményei.....	29
3.5.3.	Élőkép: az MJPEG-proxytól a go2rtc MSE-ig.....	30
3.5.4.	A kameraforgatás megvalósítása.....	34
3.5.5.	A backend és a valós idejű kommunikáció.....	34
3.5.6.	Tűz- és füstérzékelés.....	36
3.5.7.	A kezelőfelület.....	37
3.5.8.	Mobilnézet.....	42
3.5.9.	Hitelesítés és biztonsági megfontolások.....	45
3.6.	Tesztelés és mérési eredmények.....	47
3.6.1.	A mérési környezet és a módszertan.....	47
3.6.2.	Erőforrás-használat a kameraszám függvényében.....	48
3.6.3.	Detektálási sebesség: GPU-gyorsítás és processzoros futtatás.....	50
3.6.4.	Eseménykéseleltetés és az állapotfigyelés pontossága.....	51
3.6.5.	Az élőkép architektúrájának mérése.....	52
3.6.6.	A képsebesség fényfüggése – a telefonoldali korlát.....	53
3.6.7.	Megfigyelt hibamódok.....	54
3.7.	A megoldás korlátai.....	55
4.	Összegzés.....	57
4.1.	Jövőbeli fejlesztési lehetőségek.....	57
	Irodalom.....	59
5.	Mellékletek.....	60
5.1.	Nyomtatott mellékletek.....	60
5.2.	Elektronikus mellékletek.....	60

Jelmagyarázat

Jel/Rövidítés	Értelmezés
AGPL	Affero General Public License (szoftverlicenc)
AI	Artificial Intelligence (mesterséges intelligencia)
API	Application Programming Interface (alkalmazásprogramozási felület)
CPU	Central Processing Unit (központi feldolgozóegység)
DNS	Domain Name System
fMP4	Fragmented MP4 (töredezett MP4 konténerformátum)
FPS	Frames Per Second (képkocka/másodperc)
GPL	General Public License (szoftverlicenc)
GPU	Graphics Processing Unit (grafikus feldolgozóegység)
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure (titkosított HTTP)
ICE	Interactive Connectivity Establishment (kapcsolatfelépítési eljárás)
IP	Internet Protocol
JPEG	Joint Photographic Experts Group (képtömörítési szabvány)
JSON	JavaScript Object Notation (adatcsere-formátum)
JWT	JSON Web Token
MJPEG	Motion JPEG
MQTT	Message Queuing Telemetry Transport
MSE	Media Source Extensions (böngészős médialejátszási API)
NAT	Network Address Translation (hálózaticím-fordítás)
NVR	Network Video Recorder (hálózati videórögzítő)
ONNX	Open Neural Network Exchange (modellcsere-formátum)
ONVIF	Open Network Video Interface Forum
PTZ	Pan-Tilt-Zoom (forgatható, dönthető, zoomolható kamera)
REST	Representational State Transfer
RTP	Real-time Transport Protocol
RTSP	Real Time Streaming Protocol
SDP	Session Description Protocol (médialeíró formátum)
SOAP	Simple Object Access Protocol
SPA	Single Page Application (egyoldalas alkalmazás)

SSE	Server-Sent Events
TCP	Transmission Control Protocol
TPU	Tensor Processing Unit
UDP	User Datagram Protocol
URL	Uniform Resource Locator
VPN	Virtual Private Network (virtuális magánhálózat)
YOLO	You Only Look Once (objektumdetekciós modellicsalád)

Az alábbi táblázat a szakdolgozatban használt jelöléseket és rövidítéseket, valamint azok feloldását tartalmazza.

Köszönetnyilvánítás

Köszönettel tartozom mentoromnak, dr. Pletl Szilveszternek a szakdolgozat készítése során nyújtott szakmai iránymutatásért és a tervezetre adott részletes, építő jellegű észrevételekért. Köszönöm továbbá családomnak a türelmet és a támogatást – és azokat a fiókban maradt régi telefonokat, amelyek nélkül ez a rendszer nem épülhetett volna meg.

A szakdolgozat témája

A háztartásokban rendszeresen felhalmozódnak lecserélt, de hardveresen továbbra is működőképes Android okostelefonok, amelyek kamerája, Wi-Fi rádiója és akkumulátora évekkel túléli azt az időszakot, amíg a készülék a felhasználó elsődleges telefonjaként szolgál. Ezzel párhuzamosan az otthoni biztonsági kamerázás piacát jellemzően felhőalapú megoldások uralják, amelyek egyszerű üzembe helyezést kínálnak, cserébe a felvétel harmadik fél szerverére kerül, jellemzően előfizetési modellben.

A feladat egy felhőfüggetlen, helyi hálózaton működő, mesterséges intelligenciával támogatott biztonsági rendszer (SwarmCam) tervezése és megvalósítása, amely a háztartásban feleslegessé vált Android okostelefonokat – dedikált kamerahardver beszerzése nélkül – automatikusan bevonja a rendszerbe, és a felvételeket helyben, a felhasználó saját hálózatán dolgozza fel valós idejű objektum- és eseménydetekcióval.

Az újrahasznosított okostelefon a rendszerben többcélú szenzoregységként értelmezendő: a jelen munka elsődlegesen a kameraképre (gépi látásra), valamint a telefon állapotadataira (pl. akkumulátor- és töltésállapot) támaszkodik, de az architektúrát úgy kell kialakítani, hogy a készülék további beépített szenzorai (mikrofon, gyorsulásmérő, giroszkóp, fényérzékelő stb.) a későbbiekben további eseményforrásként legyenek bevonhatók.

A várható eredmény nem létező megoldások átirata vagy összehasonlító elemzése, hanem a hallgató által önállóan tervezett és megvalósított, valós hálózati környezetben tesztelt szoftverrendszer.

A szakdolgozat elkészítéséhez a hallgatónak a következő feladatokat kell elvégeznie:

1. Áttekinteni a felhőalapú és a helyben (edge) futó NVR-rendszerek (pl. Frigate) előnyeit és korlátait, valamint az ONVIF-kompatibilis IP kamerák és a fogyasztói IP-kamera-alkalmazások (pl. IP Webcam) közötti különbséget.
2. Megtervezni a rendszer összetevőkre bontott (Android telefon – AI/rögzítő réteg – üzenetközvetítő – alkalmazáslogika – felhasználói felület) architektúráját, és indokolni a komponensek közötti kommunikációs csatornák (REST, WebSocket, MQTT) megválasztását.
3. Megvalósítani a hálózaton elérhető, IP Webcam alkalmazást futtató Android eszközök automatikus, kézi konfiguráció nélküli felderítését (hálózati discovery), és a talált kamerák bekötését a választott NVR-rendszerbe.
4. Kialakítani a felhőfüggetlen, konteinerizált (Docker Compose alapú) szerverkörnyezetet, amely a videófeldolgozást, az eseménykezelést és a felhasználói felületet egyetlen, könnyen telepíthető csomagba szervezi.

5. Kiegészíteni a rendszert egy önálló, gépi tanuláson alapuló mikroszolgáltatással olyan események felismerésére (pl. tűz, füst), amelyeket az alapul szolgáló NVR általános célú objektumdetektóra natívan nem támogat.
6. Kialakítani a valós idejű, eseményvezérelt kommunikációs réteget (üzenetbróker és WebSocket kapcsolat) a kamerák állapotváltozásainak és a detektált eseményeknek a felhasználói felület felé történő továbbításához.
7. Elkészíteni a rendszer webalapú kezelőfelületét (dashboard), amely a kamerák élő képét, a rögzített eseményeket és felvételeket, valamint a rendszerbeállításokat egységes felületen teszi elérhetővé.
8. Tesztelni a rendszert valódi otthoni hálózati környezetben, több, különböző generációs Android eszközzel, és dokumentálni a mért eredményeket (erőforrás-használat a kamerák számának függvényében, átlagos esemény-késleltetés, GPU-gyorsítás hatása a detektálási sebességre).
9. Összefoglalni a megvalósított rendszer korlátait és továbbfejlesztési lehetőségeit (pl. a telefonok további beépített szenzorainak – mikrofon, gyorsulásmérő, giroszkóp – bevonása újabb eseményforrásként, dedikált ONVIF kamerák integrációja, mobil értesítések, alternatív hardveres gyorsítás).

Tudományos / szakmai terület: Elosztott rendszerek, számítógépes látás és mesterséges intelligencia (objektumdetekció), hálózati protokollok (REST, MQTT, WebSocket), konteinerizáció és szolgáltatás-orientált architektúra, webes felhasználói felületek fejlesztése.

Kiinduló irodalom:

- Frigate NVR – Documentation. Elérhető: <https://docs.frigate.video/>
- OASIS, MQTT Version 5.0 – OASIS Standard. Elérhető: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- ONVIF, ONVIF Core Specification. Elérhető: <https://www.onvif.org/specs/>
- I. Goodfellow, Y. Bengio, A. Courville – Deep Learning. MIT Press, 2016.
- Ultralytics – YOLO Docs. Elérhető: <https://docs.ultralytics.com/>
- V. Forti, C. P. Baldé, R. Kuehr, G. Bel – The Global E-waste Monitor 2020. UNU/UNITAR, ITU, ISWA, 2020.
- FastAPI – Documentation. Elérhető: <https://fastapi.tiangolo.com/>
- Docker – Docker Compose Documentation. Elérhető: <https://docs.docker.com/compose/>

1. Bevezető

1.1. A probléma leírása

Napjaink háztartásaiban rendszeresen halmozódnak fel lecserélt, de hardveresen még teljesen működőképes Android okostelefonok: kameramodul, Wi-Fi rádió és akkumulátor is van bennük, a legtöbb mégis fiókban végzi, vagy elektronikai hulladékként (e-waste) kerül megsemmisítésre. Ezzel párhuzamosan az otthoni és kisvállalati biztonsági kameráztatás piacát ma jórészt felhőalapú megoldások uralják: ezek egyszerű üzembe helyezést kínálnak, cserébe a videófolyam a gyártó szerverein landol, jellemzően előfizetéses modellben, ami adatvédelmi kockázatot és folyamatos költséget is jelent a felhasználó számára.

A dedikált, lokálisan futtatható NVR (network video recorder) szoftverek – mint a nyílt forráskódú Frigate – megoldják a felhőfüggést és az AI-alapú eseményfelismerést, de a kameraoldalt továbbra is a felhasználónak kell biztosítani, jellemzően ONVIF-kompatibilis IP kamerák megvásárlásával. Nincs olyan, egyszerűen telepíthető megoldás, amely a meglévő, használaton kívüli okostelefonokat automatikusan, kézi konfiguráció nélkül vonná be egy ilyen rendszerbe.

A SwarmCam erre a hiányra reagál: célja, hogy a hálózaton lévő, nyílt forráskódú Android IP Camera alkalmazást futtató Android eszközöket automatikusan felismerje, és egy éretté vált NVR-technológiával (Frigate) összekapcsolva egy teljesen lokális, felhőfüggetlen, AI-alapú megfigyelőhálózatot („swarmot”) hozzon létre.

1.2. A szakdolgozat céljai

A szakdolgozat fő célja a SwarmCam platform tervezése és megvalósítása, amely a fentebb vázolt problémákra ad gyakorlati megoldást. Ennek rész céljai a következők:

1. Meglévő Android okostelefonok automatikus, kézi konfiguráció nélküli integrálása IP kameraként, hálózati felderítés (discovery) segítségével.
2. Felhőfüggetlen, Docker Compose alapú szerverkörnyezet kialakítása, amely a videófeldolgozást, az eseménykezelést és a felhasználói felületet egyetlen, könnyen telepíthető csomagba szervezi.
3. A Frigate NVR beépítése a valós idejű objektum- és mozgásdetekcióhoz, kiegészítve egy önálló, YOLO-alapú mikroszolgáltatással a tűz- és füstérzékeléshez, amelyre a Frigate natívan nem ad megoldást.

4. Valós idejű, eseményvezérelt kommunikációs réteg kialakítása (MQTT a Frigate és a tűzdetektor felől, WebSocket a böngésző felé), amely a kamerák állapotváltozásait és a detektált eseményeket másodperceken belül eljuttatja a felhasználóhoz.
5. Egy modern, React alapú kezelőfelület (dashboard) fejlesztése, amely a kamerák élő képét, a rögzített eseményeket és felvételeket, valamint a rendszer beállításait egységes felületen teszi elérhetővé.

2. Szakirodalmi áttekintés

A SwarmCam tervezési döntéseit három, egymással szorosan összefüggő terület jelenlegi állapota határozza meg. Az első az otthoni videómegfigyelés uralkodó üzleti és technológiai modellje, amely a felvételt jellemzően a gyártó felhőjébe tereli, és ezzel adatvédelmi, üzemeltetési és költségbeli kötöttségeket teremt. A második a videóalapú eseményfelismerés gépi tanulási háttere: az elmúlt évtizedben az egyfázisú konvolúciós detektorok elérték azt a sebesség-pontosság arányt, amely mellett a valós idejű, több kamerát kiszolgáló detekció már egyetlen otthoni számítógépen is elvégezhető. A harmadik az elektronikai hulladék kérdése, közelebbről az a megfigyelés, hogy a háztartásokban felhalmozódó, lecserélt okostelefonok hardveresen továbbra is alkalmasak kamerafeladatra. Ez a fejezet e három területet tekinti át, majd az áttekintésből levezeti azokat a követelményeket, amelyekre a 3. fejezetben bemutatott rendszer válaszol.

Az áttekintés szándékosan nem törekszik a szakterületek teljes körű bemutatására. A cél az, hogy minden olyan fogalom és tervezési alternatíva megjelenjen benne, amelyre a megvalósítás során ténylegesen döntést kellett hozni – és amely döntés indoklása a 3. fejezetben visszakereshető.

2.1. Az otthoni videómegfigyelés technológiai környezete

2.1.1. A felhőalapú konzumer rendszerek működési modellje

A kereskedelmi forgalomban kapható okosotthon-kamerák túlnyomó többsége azonos architektúrát követ. A kamera a helyi Wi-Fi hálózatra csatlakozik, majd kimenő irányú, tartós kapcsolatot épít a gyártó felhőszolgáltatásához. A videófolyam és a mozgásérzékelés eredménye ezen a kapcsolaton keresztül hagyja el a háztartást; a felhasználó mobilalkalmazása szintén a felhőn keresztül éri el a kamerát, még akkor is, ha fizikailag ugyanabban a szobában tartózkodik. A detekció, a felvételek tárolása és az értesítések kiküldése a szolgáltató infrastruktúráján történik.

Ennek a modellnek jól azonosítható előnyei vannak. Az üzembe helyezés néhány percet vesz igénybe, és nem igényel hálózati ismereteket: nincs szükség port-továbbításra, statikus IP-címre vagy dinamikus DNS-szolgáltatásra, mert a kapcsolatot mindig a kamera kezdeményezi kifelé. A távoli hozzáférés és a frissítések a szolgáltató felelősségi körébe tartoznak, a detekció pedig nagy kapacitású központi szervereken fut, így a kamerába olcsó processzor kerülhet.

A hátrányok ugyanennek az architektúrának a következményei. A háztartás legérzékenyebb megfigyelési adata – a lakótér folyamatos videóképe – rendszeresen elhagyja a felhasználó ellenőrzési körét, és harmadik fél szerverén kerül feldolgozásra. A funkcionalitás

előfizetéshez kötött: az előzményfelvételek megőrzési ideje, az intelligens értesítések és gyakran maga a felvételkészítés is havidíjas csomag része. A rendszer működése az internetkapcsolattól és a szolgáltató életképességétől függ: az internet kiesése esetén a kamera nem érhető el, a szolgáltatás megszűnése vagy a termék támogatásának beszüntetése esetén pedig a hardver használhatatlanná válhat. Végül a felhasználó nem tudja ellenőrizni, milyen feldolgozás történik a felvételen, és azt sem, hogy az meddig marad meg.

2.1.2. Az edge, azaz helyben futó NVR alternatívája

A fenti korlátokra a helyben futó, hálózati videórögzítő (NVR) szoftverek adnak választ. Az NVR a felhasználó saját gépén – jellemzően egy alacsony fogyasztású mini PC-n, NAS-on vagy egyetlen célszerveren – fut, a kamerákat a helyi hálózaton éri el, és minden feldolgozást, tárolást és értesítést helyben végez. Ebben a modellben a videófolyam soha nem hagyja el a háztartás hálózatát, hacsak a felhasználó ezt kifejezetten nem engedélyezi.

A nyílt forráskódú Frigate ebbe a kategóriába tartozik, és több szempontból is kiemelkedik a mezőnyből [1]. Első osztályú polgárként kezeli a mesterséges intelligenciára épülő objektumdetekciót: nem utólag ráépített kiegészítőként, hanem a felvételi logika alapjaként. A rendszer először olcsó mozgásmaszkkal szűri a képkockákat, és csak a mozgást tartalmazó régiókat adja át a neurális hálónak, majd az így kapott találatokat objektumkövetéssel eseményekké fűzi össze. Az események MQTT-n publikálódnak, ami lehetővé teszi a külső integrációt anélkül, hogy a Frigate belső adatbázisát bárkinek ismernie kellene. A detektálás több gyorsítón is futtatható: processzoron, NVIDIA GPU-n (ONNX Runtime vagy TensorRT háttérrel), Google Coral TPU-n vagy Intel OpenVINO-val.

Az edge NVR modell ára az üzemeltetési komplexitás. A felhasználónak magának kell gondoskodnia a szerverről, a lemezterületről, a mentésekről és a frissítésekről, továbbá a kamerák bekötése kézi konfigurációt igényel: minden kamera RTSP-címét, felhasználónevét és jelszavát fel kell venni a konfigurációs állományba. Éppen ez az a pont, ahol a jelen dolgozat rendszere hozzáad az alapmegoldáshoz: az automatikus hálózati felderítéssel a kamerák kézi konfiguráció nélkül kerülnek be a rendszerbe.

1. táblázat: A felhőalapú és a helyben futó (edge) megfigyelési modell összehasonlítása

Szempont	Felhőalapú rendszer	konzumer	Helyben futó (edge) NVR
A videófolyam útja	Elhagyja a háztartást, a gyártó szerverén dolgozódik fel		Nem hagyja el a helyi hálózatot
Költségmodell	Hardver + havi/éves előfizetés		Egyszeri hardverköltség, előfizetés nélkül
Internetfüggés	A működéshez folyamatos		Internet nélkül is teljes

	internetkapcsolat kell	értékű
Adatmegőrzés	A szolgáltató szabályai és csomagja szerint	A felhasználó dönti el, a lemezterület a korlát
Detekció helye	Központi szerver, korlátlan kapacitással	Helyi gép, a rendelkezésre álló CPU/GPU korlátain belül
Üzembe helyezés	Néhány perc, hálózati ismeret nélkül	Szerver, tárhely és kameránkénti konfiguráció szükséges
Bővíthetőség	A gyártó ökoszisztémáján belül	Bármilyen szabványos vagy egyedi forrás integrálható

Az 1. táblázat összefoglalja a két modell közötti alapvető különbségeket. A SwarmCam egyértelműen az edge oszlopot választja, és a bal oldali oszlop egyetlen valódi előnyét – az egyszerű, konfiguráció nélküli üzembe helyezést – az automatikus felderítéssel igyekszik átemelni a jobb oldalra.

2.2. Kamerahardver: szabványos IP kamerák és fogyasztói kameraalkalmazások

2.2.1. Az ONVIF szabvány és az RTSP protokoll

A professzionális biztonságtechnikai piacon a kamerák együttműködését az ONVIF (Open Network Video Interface Forum) specifikációja szabályozza [3]. Az ONVIF SOAP-alapú webszolgáltatásokban írja le a kamera felderítését, képességeinek lekérdezését, a stream-profilok kezelését, az esemény-előfizetést és a mozgatható (PTZ) kamerák vezérlését. A gyakorlatban a legfontosabb profil a Profile S, amely a videóstreamelés és az alapvető konfiguráció közös nevezőjét rögzíti; egy Profile S kompatibilis kamera bármelyik Profile S kompatibilis NVR-hez bekötthető.

Maga a videóátvitel az ONVIF-kamerák esetében szinte kivétel nélkül RTSP-n (Real Time Streaming Protocol) történik. Az RTSP vezérlőprotokoll: a leíró (SDP) és a lejátszásvezérlő parancsokat szállítja, a tényleges médiaadat RTP-csomagokban halad, TCP- vagy UDP-alagúton. A hasznos teher többnyire H.264 (esetenként H.265) videó, amelynek legfontosabb tulajdonsága a jelen munka szempontjából, hogy a hardveresen kódolt folyam újrakódolás nélkül továbbadható, tárolható és megjeleníthető – ez az a tulajdonság, amelyre a 3. fejezetben bemutatott élőkép-architektúra épül.

Az ONVIF-kamera tehát jól definiált, szabványos és megbízható kamerahardvert jelent. Ára viszont van: egy használható minőségű ONVIF-kamera beszerzési költsége kameránként jelentkezik, ami éppen az ellenkezője annak a célnak, hogy a rendszer meglévő, feleslegessé vált eszközökből épüljön fel.

2.2.2. A fogyasztói IP-kamera alkalmazások

A másik út az, hogy a kamerát nem megvásároljuk, hanem egy meglévő okostelefonból állítjuk elő. Az Android platformon több alkalmazás is elérhető, amely a telefon kameráját hálózaton keresztül szolgáltatja ki; ezek működési elve közös: az alkalmazás miniatűr HTTP-szervert futtat a telefonon, és a kamera képét egy vagy több végponton teszi elérhetővé.

A legismertebb ilyen alkalmazás a Pavel Khlebovich által fejlesztett IP Webcam [4], amely MJPEG-folyamot, JPEG-pillanatképeket, valamint egy H.264-alapú RTSP-végpontot is kínál, továbbá HTTP-lekérésekkel vezérelhető beállításokkal (felbontás, minőség, éjjellátó mód, zseblámpa) rendelkezik. Az alkalmazás zárt forráskódú, és kizárólag a Google Play áruházból telepíthető.

A jelen munkában végül egy másik alkalmazásra esett a választás: a DigitallyRefined által fejlesztett, nyílt forráskódú (GPL) Android IP Camera alkalmazásra [12], amely az F-Droid katalógusban is elérhető. Az alkalmazás alapértelmezés szerint a 4444-es porton, HTTPS-en, ön aláírt tanúsítvánnyal szolgál ki. Végpontjai közül a rendszer szempontjából a nyers Annex-B formátumú H.264 elemi folyamat adó útvonal, az MJPEG-folyam, valamint a telefon állapotát és beállításait JSON-ban visszaadó leíró a legfontosabb. A váltás indoklását – és azt, hogy a régi alkalmazás mely telemetriai adatait kellett feláldozni érte – a 3.5.2. szakasz részletezi.

2. táblázat: Kamerahardver-alternatívák összevetése a SwarmCam követelményei szerint

Szempont	ONVIF IP kamera	Android telefon kameraalkalmazással
Beszerzési költség	Kameránkénti, új hardver	Nulla, a meglévő eszköz újrahasznosul
Szabványosság	ONVIF Profile S, RTSP – jól dokumentált	Alkalmazásfüggő, nem szabványos HTTP API
Videókodek	H.264 / H.265, hardveres enkóder	H.264, a telefon hardveres enkódere
Tápellátás	PoE vagy hálózati adapter, folyamatos	Töltőről; az akkumulátor pufferként is működik
Kiegészítő szenzorok	Nincsenek (esetleg mikrofon)	Mikrofon, gyorsulásmérő, giroszkóp, fényérzékelő, akkumulátor-állapot
Időjárásállóság	IP66/IP67 kültéri kivitel	Nincs, védett elhelyezést igényel
Élettartam-kockázat	Alacsony	Akkumulátor-degradáció, hőterhelés, elavuló Android-verzió

Automatikus felderítés	ONVIF WS-Discovery	Egyedi fingerprinting szükséges
------------------------	--------------------	------------------------------------

A 2. táblázat világossá teszi, hogy a telefonalapú node nem minden szempontból jobb az ONVIF-kameránál – időjárásállóságban és hosszú távú megbízhatóságban egyértelműen elmarad tőle. Két olyan tulajdonsága van azonban, amely a jelen munka céljának pontosan megfelel: nulla beszerzési költséggel áll rendelkezésre, és a kamerán túl további szenzorokat is hordoz, amelyek a rendszer későbbi bővítésének nyilvánvaló irányát adják.

2.3. Objektumdetekció a videóalapú megfigyelésben

2.3.1. A klasszikus mozgásdetekció és korlátai

A hagyományos riasztórendszerek mozgásérzékelése képkockakülönbségen vagy háttérmodellezésen alapul. A legegyszerűbb megközelítés két egymást követő képkocka pixelenkénti különbségét küszöböli; a fejlettebb módszerek – például a Gauss-keverékmodell (MOG2) alkalmazó háttérseparáció – pixelenként statisztikai modellt tartanak fenn a háttérrel, és az attól eltérő pixeleket jelölik előtérnek.

Ezek az eljárások olcsók és gyorsak, de nem mondanak semmit arról, hogy mi mozog. A gyakorlatban ez elviselhetetlen mennyiségű téves riasztást eredményez: a szélben mozgó ágak, a felhők árnyékának átvonulása, az éjszakai rovarok a lencse előtt, a fényváltozások és a kamerát érő rezgés mind mozgásként jelentkeznek. A rendszer használhatatlanná válik, mert a felhasználó néhány nap után figyelmen kívül hagyja az értesítéseket – ez a jelenség az oka annak, hogy a gépi látás bevezetése nem kényelmi funkció, hanem a használhatóság feltétele.

2.3.2. Konvolúciós objektumdetektorok

A mély konvolúciós hálók megjelenése alapvetően változtatta meg a helyzetet [5]. A képen látható objektumok osztályozása és lokalizálása mára rutinfeladat, és a modellcsaládok két nagy csoportra oszthatók. A kétfázisú detektorok (például a Faster R-CNN) először régiójavaslatokat állítanak elő, majd ezeket osztályozzák; pontosak, de számításigényesek. Az egyfázisú detektorok – a YOLO (You Only Look Once) család, valamint az SSD (Single Shot MultiBox Detector) – a lokalizációt és az osztályozást egyetlen előrehaladó menetben végzik el, a képet rácsra osztva és rácscellánként előrejelezve a határolókereteket, az objektumossági pontszámot és az osztályeloszlást [6].

A megfigyelési alkalmazásban az egyfázisú detektorok jelentik a gyakorlati választást, mert a képkockánkénti késleltetés közvetlenül korlátozza, hány kamerát képes egyetlen gép kiszolgálni. A YOLO-család modern változatai (v8, v11) 320×320 vagy 640×640 pixeles bemeneten dolgoznak, ami azt jelenti, hogy a detekció a rögzítéshez használt teljes felbontástól függetlenül, egy leskalázott képen fut – ez az elválasztás a jelen rendszer élőkép-architektúrájának is a kulcsa.

A gyakorlati telepítést két további tényező befolyásolja. Az egyik a modell futtatókörnyezete: a PyTorch-ban betanított modell exportálható ONNX formátumba, amely futtatható ONNX Runtime alatt [13], illetve TensorRT motorra fordítható NVIDIA GPU-n. A másik a kvantálás: a lebegőpontos súlyok 8 bites egészekre alakítása jelentősen csökkenti a memóriaigényt és a következtetési időt, és ez a feltétele annak, hogy a modell egy Google Coral típusú, célhardveres gyorsítón egyáltalán elférjen.

2.3.3. A Frigate detekciós csővezetéke

A Frigate a fenti építőelemeket egy háromlépcsős csővezetékbe rendezi, és ennek megértése nélkül a mért erőforrásadatok sem értelmezhetők helyesen. Az első lépcső a mozgásmaszk: a rendszer minden beérkező képkockán olcsó, klasszikus mozgásdetekciót futtat. A második lépcső csak akkor indul el, ha a mozgásmaszk talált valamit – ekkor a mozgó régiók köré illesztett kivágatokat adja át a neurális hálónak. A harmadik lépcső az objektumkövetés: a képkockánként visszakapott találatokat időben összefűzi, így egyetlen, a kamera előtt átsétáló személyből egyetlen esemény keletkezik, nem pedig kétszáz különálló detektálás.

Ennek a felépítésnek két, a mérések szempontjából fontos következménye van. Az egyik, hogy a detektor terhelése nem a kamerák számával, hanem a képen zajló mozgás mennyiségével arányos: egy mozdulatlan képet mutató kamera gyakorlatilag nem terheli a neurális hálót. A másik, hogy a rendszer két, egymástól független paraméterrel szabályozható – a detekciós képsebességgel (detect.fps) és a detekciós felbontással –, amelyek a rögzített felvétel minőségét egyáltalán nem befolyásolják.

2.4. Kommunikációs minták elosztott rendszerekben

Egy több komponensből álló, valós idejű rendszerben a komponensek közötti kommunikáció megválasztása önálló tervezési feladat. Az alábbi négy minta mindegyikének megvan a maga jól körülhatárolt szerepe, és a jelen rendszer mind a négyet használja – de mindegyiket csak arra, amire való.

A REST a HTTP kérdés-válasz modelljére épülő, szinkron erőforrás-hozzáférés. Állapotmentes, jól gyorsítótárazható, minden eszközön és nyelvben triviálisan implementálható, és a hibakezelése a HTTP-státuszkódokra épülhet. Cserébe a szerver nem tud magától üzenni: minden interakciót a kliensnek kell kezdeményeznie. Olyan műveletekhez alkalmas, ahol a kliens tudja, mikor van szüksége az adatra – például egy eseménylista lekérésére vagy egy beállítás mentésére.

A WebSocket a HTTP-kapcsolat protokollváltásával (Upgrade) létrehozott, tartós, kétirányú csatorna. Az első kézfogás után mindkét fél bármikor küldhet üzenetet, keretként néhány bájt fejléc-többséggel. Pontosan arra való, amire a REST nem: a szerver által kezdeményezett, folyamatos állapotfrissítésre. Költsége, hogy a kapcsolat állapotot hordoz, tehát a szervernek nyilván kell tartania a kapcsolódott klienseket, és kezelnie kell a bontást és az újracsatlakozást.

Az SSE (Server-Sent Events) a WebSocket egyszerűsített, egyirányú rokona: egyetlen, nyitva tartott HTTP-válaszfolyamon keresztül a szerver szöveges eseményeket küld a kliensnek. Mivel közönséges HTTP-válasz, minden fordított proxyn átmegy, és a böngésző beépített EventSource API-jával kezelhető. Olyan feladatokra ideális, ahol a kliens csak fogad – például egy hosszan futó művelet előrehaladásának megjelenítésére.

Az MQTT könnyűsúlyú, publish-subscribe modellű üzenetprotokoll, amelyet eredetileg alacsony sávszélességű telemetriai alkalmazásokra terveztek [2]. A küldő és a fogadó nem ismeri egymást: a küldő egy témába (topic) publikál, a bróker pedig eljuttatja az üzenetet minden feliratkozóhoz. Ez a szétcsatolás a jelen rendszerben nem elméleti előny, hanem konkrét képesség: két, egymást nem ismerő eseményforrás – a Frigate és a saját fejlesztésű tűzdetektor – ugyanannak a fogyasztónak, a backendnek küldhet eseményt anélkül, hogy bármelyikük tudna a másiktól.

3. táblázat: A kommunikációs minták és jellemzőik

Minta	Irány	Kapcsolat	Tipikus felhasználás
REST / HTTP	Kliens kérdez, szerver válaszol	Rövid életű, állapotmentes	Erőforrás-lekérés és -módosítás
WebSocket	Kétirányú	Tartós, állapotot hordoz	Folyamatos állapotfrissítés, riasztás
SSE	Csak szerver → kliens	Tartós HTTP-válaszfolyam	Hosszan futó művelet előrehaladása
MQTT	Publikáló → bróker → feliratkozók	Tartós, brókeren keresztül	Szétcsatolt, aszinkron eseményközvetítés

A 3. táblázat mintáit a 3.3. szakasz köti hozzá a rendszer konkrét adatfolyamaihoz.

2.5. Konténerizáció és szolgáltatásorientált üzemeltetés

Egy öt szolgáltatásból álló rendszer kézi telepítése – Python-környezet, Node-eszközlánc, MQTT-bróker, NVR, webkiszolgáló, GPU-illesztők – a felhasználó számára vállalhatatlan terhet jelentene, és minden gépen más eredményt adna. A konténerizáció erre ad választ: az egyes szolgáltatások a függőségeikkel együtt, egymástól izoláltan, de a gazdagép kerneljét megosztva futnak. A Docker Compose több konténer együttes leírását teszi lehetővé egyetlen deklaratív állományban [9] – a képfájlok, a környezeti változók, a kötetek, a hálózatok és az indítási sorrend egy helyen –, így a teljes rendszer egyetlen paranccsal

elindítható. Ez nem kényelmi kérdés: a feladatkiírás kifejezetten előírja, hogy a szerverkörnyezet „egyetlen, könnyen telepíthető csomagba” legyen szervezve.

A konténerhálózatok közül kettő releváns. A bridge mód esetén a konténer saját hálózati névtérben, saját IP-címmel fut, és a gazdagéphez portleképezésen keresztül kapcsolódik; ez az alapértelmezett, jól izolált megoldás. A host mód esetén a konténer közvetlenül a gazdagép hálózati névtérét használja: nincs NAT, nincs portleképezés, és a konténer ugyanúgy látja a helyi hálózatot, mint a gazdagép. Videóalkalmazásban ez utóbbinak konkrét haszna van – a hálózati felderítés és a helyi hálózati kamerák alacsony késleltetésű elérése egyaránt ezt kívánja –, ára pedig az izoláció csökkenése.

2.6. Elektronikai hulladék és a mobileszközök újrahasznosítása

A mobiltelefonok gyors modellváltása globálisan is mérhető környezeti terhet jelent. A Global E-waste Monitor 2020 adatai szerint a világ éves elektronikaihulladék-termelése az 50 millió tonnás nagyságrendben mozog, és növekvő tendenciát mutat, miközben a szabályozott, dokumentált újrahasznosítás aránya ennek töredékét fedi le [7]. A kisméretű informatikai eszközök – ide tartozik az okostelefon is – ezen belül külön kategóriát alkotnak, mert nagy értékű anyagokat (ritkaföldfémeket, aranyat, rezet) tartalmaznak, ugyanakkor a méretük miatt gyakran a kommunális hulladékba vagy a fiók mélyére kerülnek.

A jelen munka szempontjából azonban nem az anyagvisszanyerés, hanem az újrahasználat a releváns. Egy három-négy éves okostelefon hardvere lényegében sértetlen: a kameramodul felbontása és fényereje bőven meghaladja a belépő szintű biztonsági kamerakét, a Wi-Fi rádió stabil, a rendszerchip pedig hardveres H.264 enkódert tartalmaz, amely a videótömörítést a processzor terhelése nélkül végzi el. A készülék jellemzően nem azért kerül lecserélésre, mert bármelyik alkatrésze meghibásodott, hanem mert az akkumulátora gyengült, a szoftverfrissítések elmaradtak, vagy egyszerűen megjelent egy vonzóbb modell.

Kamerafeladatban ezek a hiányosságok többnyire nem korlátozóak. Az akkumulátor-degradáció nem számít, ha a készülék állandó tápellátáson üzemel – sőt, a maradék kapacitás beépített szünetmentes tápként viselkedik, és rövid áramkimaradást is átvészel. Az elavult Android-verzió sem akadály, amennyiben a kamerát kiszolgáló alkalmazás támogatja; a jelen rendszerben használt alkalmazás Android 7.0 vagy újabb rendszert igényel, ami a gyakorlatban a 2016 után gyártott készülékeket jelenti.

A megközelítés valós korlátai máshol vannak. Az első a hőterhelés: a folyamatos videó kódolás melegíti a készüléket, és a huzamos, magas hőmérsékleten tartott lítiumion-cella degradációja felgyorsul, szélsőséges esetben a cella felfűvódhat. A második az időjárás: telefon önmagában nem helyezhető ki védtelen kültéri környezetbe. A harmadik a szoftveres megbízhatóság: az Android agresszív energiagazdálkodása leállíthatja a háttérben futó alkalmazást, és a kameraszolgáltatás elakadása esetén távoli újraindításra nincs szabványos

mód – ez a jelen rendszerben ténylegesen megfigyelt hibamód, amelyet a 3.6.7. szakasz dokumentál.

Összefoglalva: az áttekintés három követelményt jelöl ki a következő fejezetben bemutatott rendszer számára. A megfigyelési adat nem hagyhatja el a helyi hálózatot; a használhatóság feltétele a gépi látás alapú eseményszűrés; a kamerahardver költsége pedig nem terhelheti a felhasználót – aminek az az ára, hogy a kamerák bekötését, állapotfigyelését és hibatűrését a rendszernek magának kell megoldania.

3. Problémamegoldás

Ez a fejezet a SwarmCam rendszer tervezését és megvalósítását mutatja be. A felépítése követi a fejlesztés tényleges menetét: először a követelményekből és az azokból következő tervezési kényszerekből indul ki, majd ismerteti a rendszer rétegzett architektúráját és a rétegek közötti kommunikációs csatornák megválasztásának indoklását. Ezt követi a konténerizált üzemeltetési környezet leírása, majd a legfontosabb komponensek megvalósításának részletes tárgyalása. A fejezet a valós hálózaton elvégzett mérésekkel és azok elemzésével zárul, végül összefoglalja a megoldás tudatosan vállalt korlátait.

A bemutatás során minden lényeges tervezési döntésnél megjelenik az elvetett alternatíva is, és annak indoklása, hogy miért esett a választás a megvalósított megoldásra. Több esetben – így az élőkép-architektúra és a detektorgyorsítás esetében – ez az indoklás mérési eredményekre támaszkodik, amelyeket a 3.6. szakasz közöl.

3.1. Követelmények és tervezési kényszerek

A rendszerrel szemben támasztott funkcionális követelmények közvetlenül a feladatkiírásból következnek. A rendszernek automatikusan, kézi konfiguráció nélkül fel kell derítenie a helyi hálózaton elérhető Android kamera-node-okat, és be kell kötnie őket a rögzítő rétegbe. Valós idejű objektumdetekciót kell futtatnia a kamerák képén, és az így keletkező eseményeket – a felvétellel és a pillanatképpel együtt – böngészhetővé kell tennie. Ki kell egészülnie egy olyan eseményosztály felismerésével, amelyet az alapul szolgáló NVR általános célú objektumdetektora nem támogat; a választott osztály a tűz és a füst. Folyamatosan figyelnie kell a node-ok állapotát – elérhetőség, akkumulátorszint, Wi-Fi jelerősség –, és minden állapotváltozást azonnal meg kell jelenítenie a felhasználói felületen. Végül mindezt egyetlen, egységes webes kezelőfelületen kell elérhetővé tennie.

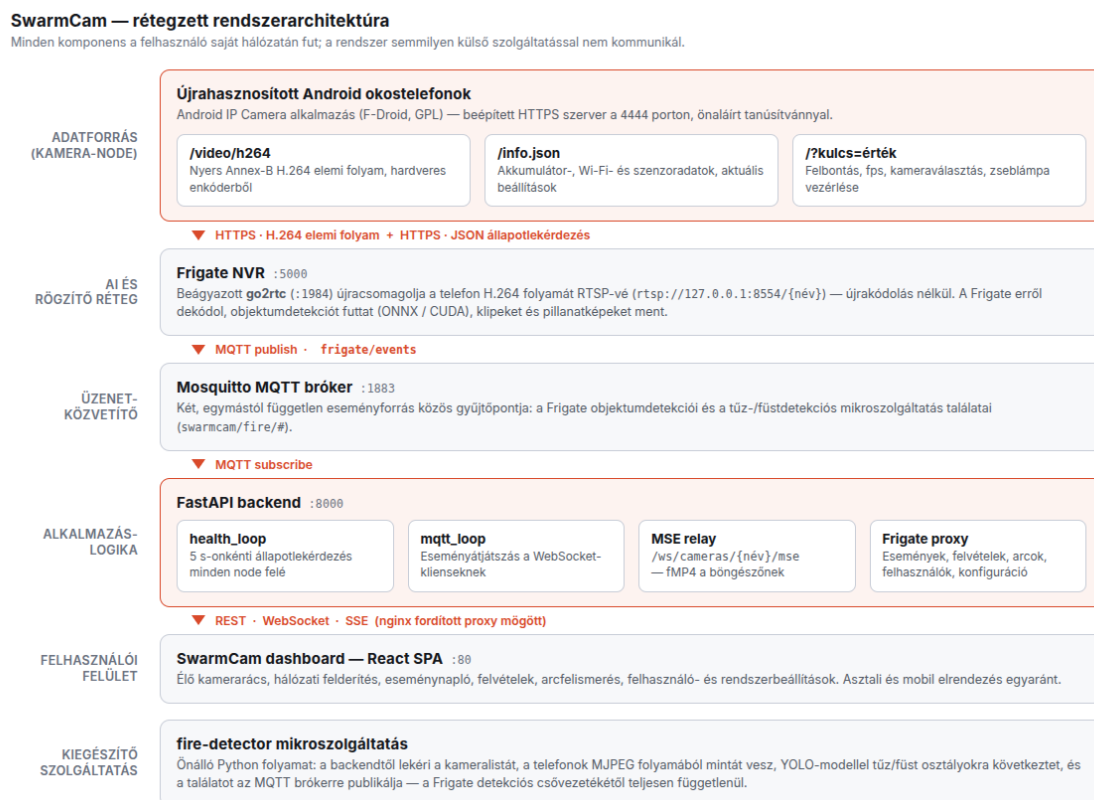
A nem funkcionális követelmények részben ugyaninnen, részben az áttekintő fejezet következtetéseiből erednek. A legfontosabb a felhőfüggetlenség: a rendszer egyetlen komponense sem kommunikálhat külső szolgáltatással, és internetkapcsolat nélkül is teljes értékűen kell működnie. Ezt követi a telepíthetőség (a teljes szerveroldal egyetlen paranccsal álljon fel), a valós idejűség (az állapotváltozások és az események lekérdezés nélkül, a szerver kezdeményezésére, másodperces nagyságrendben jelenjenek meg), a kímélő erőforrás-használat (egyetlen, alacsony fogyasztású gép szolgáljon ki több kamerát), végül a hibatűrés: egyetlen node kiesése – ami akkumulátorról vagy Wi-Fi-ről üzemelő telefonoknál nem kivétel, hanem üzemszerű állapot – nem befolyásolhatja a többiek működését.

Ezekből három olyan tervezési kényszer vezethető le, amely a teljes architektúrát meghatározza. Az első: mivel a kamera-node egy fogyasztói alkalmazást futtató telefon, nem

pedig szabványos ONVIF-eszköz, a felderítés nem építhet szabványos protokollra, hanem egyedi ujjlenyomat-vizsgálatot igényel. A második: mivel a node bármikor eltűnhet a hálózatról, a rendszernek minden komponensében fel kell készülnie a forrás átmeneti hiányára – ez az élőkép-, a rögzítő- és az állapotfigyelő ágat egyaránt érinti. A harmadik: mivel a detekció számításigénye a limitáló tényező, a videófeldolgozási láncban minden felesleges kódolási és dekódolási lépést el kell kerülni.

3.2. A SwarmCam architektúrája

A rendszer öt egymástól jól elkülönített, konténerizált szerveroldali szolgáltatásból, valamint a hálózaton szétszórt kamera-node-okból áll. Az architektúra rétegzett: minden réteg csak a közvetlenül alatta és fölötte lévővel kommunikál, jól definiált interfészen keresztül. A rétegek elrendezését és a köztük áramló adat jellegét az 1. ábra foglalja össze.



1. ábra: A SwarmCam rétegzett rendszerarchitektúrája. Minden komponens a felhasználó saját hálózatán fut; a rendszer semmilyen külső szolgáltatással nem kommunikál.

Az 1. ábra legfelső rétege az adatforrás: az Android IP Camera alkalmazást futtató, újrahasznosított okostelefonok. Minden node három interfészt kínál, amelyek közül a rendszer mindhármát használja. A nyers, Annex-B formátumú H.264 elemi folyam adja a videót; ezt a telefon rendszerchipjének hardveres enkódere állítja elő, tehát a tömörítés nem terheli a telefon processzorát. A JSON formátumú leíró a telefon állapotát – akkumulátorszint, Wi-Fi jelerősség – és aktuális beállításait adja vissza, egyben a felderítés ujjlenyomataként is

szolgál. A lekérdezési paraméterekkel vezérelhető beállítási felület a felbontás, a képsebesség, az aktív kameraszenzor és a zseblámpa távoli állítását teszi lehetővé.

A második réteg az AI és rögzítő réteg, amelyet a Frigate NVR alkot. Fontos részlet, hogy a Frigate nem közvetlenül a telefonhoz csatlakozik: a beágyazott go2rtc komponens [14] veszi át a telefon H.264 folyamát, és helyi RTSP-forrásként szolgálja ki újra. A Frigate ezt a helyi RTSP-forrást dekódolja, futtatja rajta az objektumdetekciót, és menti a klipeket és pillanatképeket. A közbeiktatott go2rtc három konkrét problémát old meg, amelyeket a 3.5.3. szakasz részletez.

A harmadik réteg az üzenetközvetítő: a Mosquitto MQTT bróker. Két, egymást nem ismerő eseményforrás – a Frigate és a saját fejlesztésű tűzdetektor – közös gyűjtőpontja. A negyedik réteg az alkalmazáslogika, amelyet a FastAPI backend valósít meg [8]: ez felel az állapotfigyelésért, az eseményátjátszásért, az élőkép-relayért, valamint a Frigate REST API-jának proxyzásáért. Az ötödik réteg a felhasználói felület: a React alapú, egyoldalas dashboard alkalmazás [11].

Az architektúrában külön helyet foglal el a tűz- és füstdetekciós mikroszolgáltatás. Ez nem illeszkedik a fenti rétegsorba, mert nem lineárisan kapcsolódik: a kameralistát a backendtől kéri le, a képet közvetlenül a telefonoktól veszi, az eredményt pedig az MQTT brókerre publikálja. A szolgáltatás így az architektúra bármelyik pontjának kiesését túléli, és nem befolyásolja a Frigate detekciós csővezetékét.

Indokolt kitérni arra, hogy miért nem készült saját detekciós motor. A valós idejű, több kamerát kiszolgáló, megbízható objektumdetekció megvalósítása – a mozgásmaszkkal, az objektumkövetéssel, a felvételkezeléssel és a hardveres gyorsítók támogatásával együtt – önmagában egy teljes szakdolgozat terjedelmét kitöltené, és a rendelkezésre álló nyílt forráskódú megoldásnál gyengébb eredményt hozna. A jelen munka hozzájárulása nem a detekciós motor újraírása, hanem az a köré épített integrációs réteg, amely a Frigate-et egy telefonokból álló, dinamikusan változó kameraparkkal használhatóvá teszi: a hálózati felderítés, az állapotfigyelés, a valós idejű eseményközvetítés, az élőkép-architektúra, a tűzdetekciós kiegészítés és a teljes kezelőfelület. Ezek mindegyike a szerző önálló tervezése és megvalósítása.

3.3. A hálózati és kommunikációs réteg tervezése

A komponensek közötti adatcsere négy, egymástól eltérő célú csatornán zajlik. A csatornák megválasztása nem eklektikusság, hanem az egyes adatfolyamok eltérő jellegének következménye; a 2.4. szakaszban áttekintett minták mindegyike arra a feladatra került, amelyre való.

4. táblázat: A SwarmCam kommunikációs csatornái és megválasztásuk indoklása

Csatorna	Szereplők	Szállított adat	A választás indoka
----------	-----------	-----------------	--------------------

REST / HTTP	Frontend ↔ backend, ↔ Frigate	Kameralista, események, felvételek, arcok, felhasználók, konfiguráció	Szinkron, állapotmentes erőforrás-hozzáférés; a kliens tudja, mikor van szüksége az adatra
WebSocket	Backend → frontend	Kamerastátusz, detektált események, riasztások	A szerver kezdeményezi a frissítést; egyetlen tartós kapcsolat minden eseménytípusra
WebSocket (bináris)	go2rtc → backend → böngésző	fMP4 videószegekmenek (élőkép)	A meglévő nginx- és hitelesítési útvonalon halad; nincs szükség ICE-tárgyalásra és UDP-portra
SSE	Backend → frontend	A hálózati felderítés élő naplója	Egyirányú, hosszan futó folyamat előrehaladása; közös HTTP-válasz, minden proxyn átmege
MQTT	Frigate és fire-detector → bróker → backend	Objektumdetekció és tűz-/füstesemények	Két, egymást nem ismerő forrás szétcsatolt közvetítése; a bróker átmeneti backend-kiesést is túlél

A 4. táblázat kapcsán három döntés érdemel külön magyarázatot.

Az első: miért egyetlen WebSocket kapcsolat szolgálja ki mind a kamerastátuszt, mind a detektált eseményeket, mind a riasztásokat. Az alternatíva az lett volna, hogy minden üzenettípus saját végpontot kap. Ez tisztább szeparációt adna, de háromszor annyi kapcsolatot jelentene böngészőnként, háromszoros újracsatlakozási logikával. Mivel mindhárom üzenettípus ugyanannak a felületnek szól, és a forgalom mennyisége elhanyagolható, a type mezővel megkülönböztetett, közös csatorna egyszerűbb és megbízhatóbb.

A második: miért SSE és nem WebSocket a felderítés naplója. A felderítés egyirányú folyamat – a kliens elindítja, majd csak fogad –, és jellemzően néhány tíz másodpercig tart. Egy tartós, kétirányú kapcsolat felépítése és bontása ehhez felesleges. Az SSE további előnye, hogy közöségi HTTP-válaszfolyam, tehát a fordított proxy konfigurációjában nem igényel külön Upgrade-kezelést.

A harmadik: miért MQTT és nem a Frigate eseményvégpontjának lekérdezése. Az MQTT push-alapú, tehát az esemény milliszekundumokon belül eljut a backendhez, míg a lekérdezés a lekérdezési periódus felével megnövelné az átlagos késleltetést, és üresjáratit terhelést jelentene. A döntő érv azonban a szétcsatolás: a bróker miatt a Frigate és a tűzdetektor egyáltalán nem tud egymásról és a backendről sem, továbbá a backend újraindítása nem igényel semmilyen összehangolást a többi komponenssel.

Egy technikai részlet, amely a fenti csatornákat a gyakorlatban összeköti: a dashboardot kiszolgáló nginx konfigurációjában kizárólag a /ws/ útvonal továbbítja a WebSocket Upgrade fejléct, az /api/ nem. Ezért minden WebSocket végpont – a státuszcsatorna és az élőképrelay egyaránt – ezen az előtagon érhető el. Ez nem esztétikai kérdés: az előtag elhagyása esetén a kapcsolat közöségi HTTP-kérésként érkezne a backendhez, és azonnal lezárulna.

3.4. Konténerizáció és üzemeltetés

A szerveroldali infrastruktúra telepítését és izolálását a Docker Compose végzi, egyetlen parancsból induló, öt konténerből álló együttesként. Az 5. táblázat összefoglalja a konténereket és a köztük fennálló legfontosabb különbségeket.

5. táblázat: A SwarmCam Docker-konténerei

Konténer	Alap / build	Hálózat	Feladat
frigate	ghcr.io/blakeblackshear/frigate:stable-tensorrt	host, NVIDIA runtime	AI-alapú NVR, go2rtc restream, felvételkezelés
mosquitto	eclipse-mosquitto:2	bridge (swarmcam)	MQTT üzenetbróker
swarmcam-backend	saját build (FastAPI)	host	REST/WebSocket API, felderítés, állapotfigyelés
swarmcam-dashboard	saját build (nginx + React)	host	A statikus frontend kiszolgálása, fordított proxy
swarmcam-fire-detector	saját build (YOLO + paho-mqtt)	host	Tűz- és füstdetekció

A táblázat két sora tervezési döntést rejt. Az egyik a `network_mode`: host használata négy konténernél. A Frigate esetében ez azért szükséges, mert a kamerák RTSP-forgalmát a lehető legkisebb késleltetéssel kell feldolgoznia, és a Docker NAT-rétege ebbe fölöslegesen szólna bele. A backend esetében azért, mert így a Frigate-et és a `go2rtc`-t közvetlenül a `localhost` címen éri el, konténer-DNS nélkül, továbbá – és ez a fontosabb – így képes a helyi hálózat teljes alhálózatát végigpásztázni a felderítés során. Bridge módban a konténer saját, izolált hálózati névtérben lenne, ahonnan a fizikai alhálózat hosztjai nem érhetők el közvetlenül. A host mód ára az izoláció csökkenése: a konténerek portjai közvetlenül a gazdagépen jelennek meg. Mivel a rendszer definíció szerint kizárólag a helyi hálózaton üzemel, ez vállalható kompromisszum.

A másik döntés a Docker-vezérlőaljzat (`/var/run/docker.sock`) becsatolása a backend konténerbe. Erre azért van szükség, mert a Frigate a konfigurációs állományát kizárólag induláskor olvassa be: ha a felhasználó a beállítások oldalon megváltoztatja a dekóder típusát, a detekciós képsebességet vagy egy kamera orientációját, a változás csak a Frigate újraindításával lép életbe. A becsatolt aljzat révén a backend maga adhatja ki az újraindítási parancsot, így a felhasználónak nem kell terminálhoz nyúlnia. Ez a megoldás biztonsági szempontból figyelmet érdemel – a Docker-aljzathoz való hozzáférés gyakorlatilag a gazdagép feletti teljes ellenőrzést jelenti –, és kizárólag azért vállalható, mert a rendszer zárt, helyi hálózaton, egyetlen felhasználó ellenőrzése alatt üzemel.

A perzisztens adat három helyen él. A Frigate saját adatbázisa és a felvételek egy Docker-köteten, illetve a gazdagép fájlrendszerére csatolt könyvtárban; a kameraaliasok – vagyis a felhasználó által adott, emberi olvasásra szánt nevek – egy JSON állományban; a tűzesemények pedig egy különálló SQLite adatbázisban. A backend belső állapota ezzel szemben szándékosan kizárólag a memóriában él: a kameralista, az állapot-gyorsítótár és a kapcsolódott WebSocket-kliensek halmaza modulszintű változókból. Indításkor a kameralista a Frigate konfigurációs állományából épül újra, méghozzá a `go2rtc.streams` szekcióban szereplő URL-ekből visszafejtve, mert a Frigate a saját kameraszekciójában már csak a belső RTSP-címet látja. Ez a megoldás azért választható, mert a rendszer egyetlen csomóponton, kis kameraszámmal üzemel, és a valóban perzisztálendő adatokat – a felvételeket és az eseményeket – a Frigate úgyis a saját adatbázisában kezeli. Egy önálló adatbázisréteg bevezetése ezen a méreten csak duplikációt és szinkronizációs hibalehetőséget adna hozzá a rendszerhez.

3.5. Implementáció

3.5.1. Automatikus hálózati felderítés

A felderítés a `discovery/discovery.py` önálló szkriptben valósul meg, amelyet a backend a `/api/discover/stream` végponton keresztül külön folyamatként indít el. Az önálló szkript mellett három érv szólt. Az első a tesztelhetőség: a felderítés parancssorból, a teljes rendszer

elindítása nélkül is futtatható és hibakereshető. A második az izoláció: egy több száz párhuzamos socketet nyitó pásztázás nem terheli a backend eseményhurokát. A harmadik a naplózhatóság: a szkript a folyamat előrehaladását a hibakimenetre írja, amit a backend soronként továbbít a felületnek Server-Sent Events formájában, így a felhasználó élőben követheti a pásztázást.

Maga a felderítés kétlépcsős ujjlenyomat-vizsgálat. Az első lépcső egy gyors TCP-portpásztázás: a szkript a megadott – vagy a gazdagép hálózati interfészeiből automatikusan kiolvasott – alhálózat minden címén megkísérel kapcsolatot nyitni a 4444-es porton, 128 párhuzamos szálon, egy másodperces időkorláttal. Ez a lépés egy /24-es alhálózat 254 címét néhány másodperc alatt szűri le.

```
def _port_open(ip: str, port: int, timeout: float) -> bool:
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.settimeout(timeout)
        return s.connect_ex((ip, port)) == 0
```

A második lépcső azonosítja, hogy a nyitott port mögött valóban a keresett kameraalkalmazás fut-e. A szkript HTTPS-en lekéri az eszköz /info.json leíróját, és az ujjlenyomatot a válasz szerkezetére alapozza: az Android IP Camera válasza mindig tartalmazza a cameras és a settings kulcsot egyszerre, ami a helyi hálózaton előforduló egyéb HTTPS-szolgáltatásoktól megbízhatóan megkülönbözteti.

```
def _fetch_info(ip: str, port: int) -> dict | None:
    resp = requests.get(f"https://{ip}:{port}/info.json",
                        timeout=HTTP_TIMEOUT, verify=False)
    if resp.status_code == 200:
        data = resp.json()
        if "cameras" in data and "settings" in data:
            return data
    return None
```

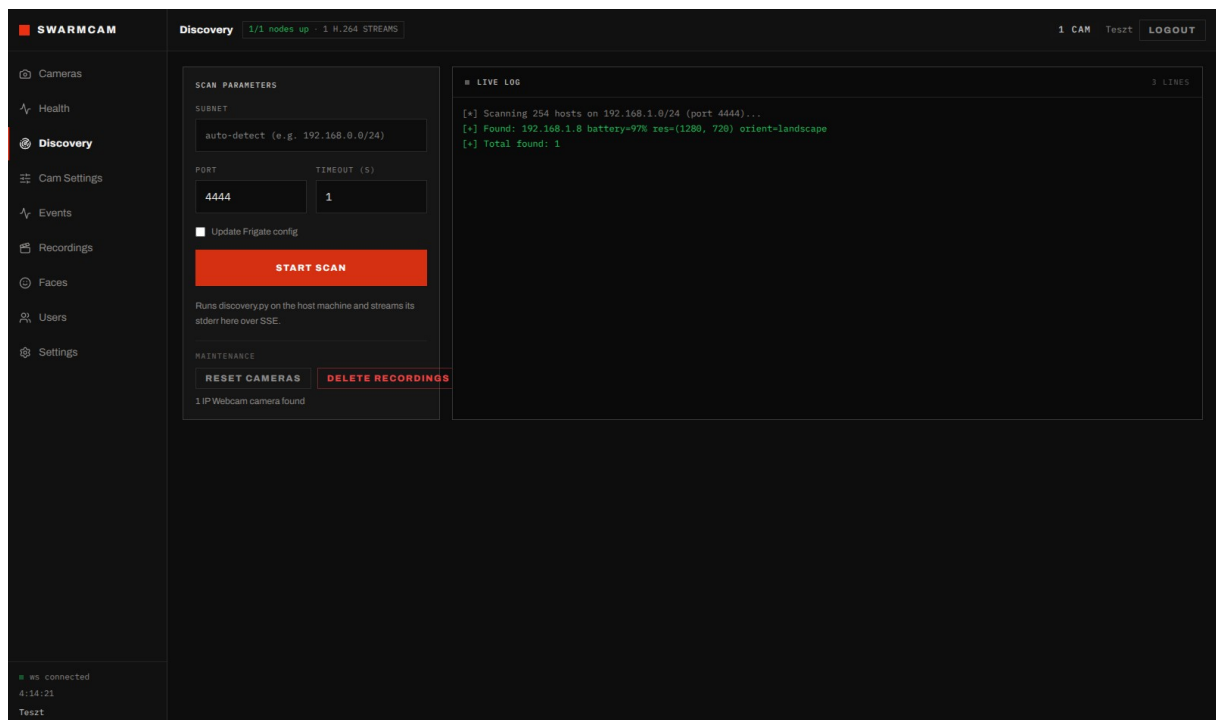
A HTTPS-kérés tanúsítványellenőrzés nélkül (verify=False) történik. Ez tudatos döntés, amelynek három indoka van. Egyrészt az alkalmazás alapértelmezés szerint HTTPS-t szolgál ki, és a titkosítás kikapcsolása minden telefonon külön kézi lépést igényelne – éppen azt a kézi konfigurációt, amelynek elkerülése a rendszer célja. Másrészt a tanúsítványt az alkalmazás az első induláskor maga állítja elő; tanúsítóhatóság nem létezik, így az ellenőrzésnek nem lenne mit ellenőriznie. Harmadrészt a forgalom kizárólag a helyi hálózaton közlekedik. Fontos hangsúlyozni, hogy a verify=False a titkosítást nem kapcsolja ki, csak a szervertípus hitelesítését hagyja el.

A leíróból a szkript kiolvassa az akkumulátorszintet, a Wi-Fi jelerősséget, az aktív kameraszenzort és annak felbontását, valamint az orientációt. A felbontás meghatározása külön figyelmet igényel: a beállított érték lehet konkrét méret, de lehet auto vagy max kulcsszó is; utóbbi esetben a szkript az aktív szenzor támogatott felbontásai közül a legnagyobbat választja tartaléknak.

A megtalált eszközök determinisztikus nevet kapnak: cam_ előtag után az IP-cím, pontok helyett aláhúzásjelekkel – például cam_192_168_1_8. A determinisztikus névképzés két gyakorlati előnyt ad. Az egyik, hogy ugyanaz a telefon újraindítás vagy ismételt felderítés

után is ugyanazt a belső azonosítót kapja, tehát a hozzá tartozó felvételek és beállítások nem szakadnak el tőle. A másik, hogy a névséma a kameraprotokoll cseréjét is túlélte: mivel a név csak az IP-címtől függ, a felhasználó által adott, emberi olvasásra szánt nevek – amelyek egy külön JSON állományban élnek – migráció nélkül maradtak érvényben.

A felderítés opcionálisan a Frigate konfigurációs állományát is frissíti. Ez két szekció együttes írását jelenti: a `cameras:` szekcióba a belső RTSP-forrásra hivatkozó bejegyzés kerül, a `go2rtc.streams:` szekcióba pedig a telefon tényleges HTTPS-címe. A két szekció szétválasztása a 3.5.3. szakaszban tárgyalt élőkép-architektúra következménye.



2. ábra: Hálózati felderítés futás közben. A jobb oldali panel a felderítő szkript hibakimenetét mutatja élőben, Server-Sent Events csatornán érkező; a bal oldali panelen a pásztázás paramétereit állíthatók.

A 2. ábra a felderítés felületét mutatja egy befejezett pásztázás után. A napló első sora a pásztázott alhálózatot és portot jelzi, a második a megtalált node-ot az azonnal kiolvasott állapotadatokkal együtt. A bal oldali panel „Maintenance” szekciója két karbantartó műveletet is kínál: a kameralista teljes törlését és a felvételek törlését.

3.5.2. A kamera-protokoll cseréje és annak következményei

A rendszer eredetileg az IP Webcam alkalmazásra épült, és a fejlesztés egy későbbi szakaszában került át a nyílt forráskódú Android IP Camera alkalmazásra. A váltás oka nem technikai hiba volt, hanem architektúráis következetlenség: egy olyan rendszerben, amelynek deklarált célja a teljes felhő- és szolgáltatófüggetlenség, idegen elem egy zárt forrású, kizárólag a Google Play áruházból telepíthető komponens. A gyakorlatban ez konkrét akadályt is jelentett: az áruház nem minden készüléken nyitja meg az alkalmazás oldalát – gyártói szolgáltatások hiánya és regionális korlátozások miatt, függetlenül a

készülék korától –, tehát nem minden kéznél lévő telefon volt bevonható a rendszerbe. Ez pontosan az a felhasználói élmény, amelynek elkerülése a projekt kiindulópontja.

A döntés teljes cserére szült, nem párhuzamos támogatásra. A párhuzamos támogatás megduplázza volna a felderítési és állapotfigyelési kódutakat, két külön tesztelendő ággal, miközben a kiváltandó probléma – a régi alkalmazás korlátozott elérhetősége – megmaradt volna. A 6. táblázat a két protokoll leképezését foglalja össze.

6. táblázat: A kamera-protokoll leképezése az alkalmazáscsere során

Funkció	IP Webcam (régi)	Android IP Camera (új)
Alapértelmezett port	8080 (HTTP)	4444 (HTTPS, önálírt tanúsítvány)
Ujjlenyomat	/status.json, „curvals” kulcs	/info.json, „cameras” + „settings” kulcs
MJPEG élőkép	/videofeed	/video/mjpeg
Videóforrás a rögzítéshez	Közvetlen RTSP-végpont	/video/h264 nyers folyam, go2rtc újracsomagolással
Zseblámpa	/enabletorch, /disabletorch	/?torch=on / ?torch=off
Beállításírás	Kulcsenként külön kérés	Egyetlen kérés több paraméterrel
Licenc és terjesztés	Zárt forrás, csak Play áruház	GPL, F-Droid és forráskód

A csere tudatosan vállalt veszteségekkel járt, amelyeket a dolgozat őszintén rögzít. Az új alkalmazás nem szolgáltatja a JPEG-minőség és az éjjellátó mód beállítását, a szabad tárhelyre, az akkumulátor hőmérsékletére és feszültségére vonatkozó adatokat, valamint az élő nézők számát. Ezek a funkciók ennek megfelelően kikerültek a rendszerből. A legérzékenyebb veszteség a töltő csatlakoztatásáról és lecsatlakoztatásáról szóló riasztás: a régi protokoll jelezte a töltési állapotot, így a rendszer figyelmeztetni tudott arra, ha egy folyamatosan üzemelő telefon táplálása megszűnt. Ez az adatforrás az új alkalmazásban nem érhető el.

Cserébe új képességek jelentek meg. Az új alkalmazás visszaadja a Wi-Fi jelerősséget dBm-ből származtatott százalékos formában, támogatja a nagyítást, az expozíció- és kontrasztállítást, továbbá a videót natív, hardveres H.264 enkóderrel állítja elő. Ez utóbbi a rendszer szempontjából a legfontosabb: ez teszi lehetővé, hogy a videó a telefontól a böngészőig egyetlen újrakódolás nélkül jusson el.

3.5.3. Élőkép: az MJPEG-proxytól a go2rtc MSE-ig

Az élőkép megvalósítása a projekt egyik legtöbbet átdolgozott része, és jól illusztrálja, hogyan vezethet egy mérés architektúraváltáshoz. Az első megvalósítás a Frigate

hibakeresési MJPEG-folyamát proxyzta a böngészőnek. Ez a megközelítés egyszerű volt és működött, de három szerkezeti problémával járt.

Az első, hogy a Frigate ezen a végponton a *detekciós* képkockákat kódolja JPEG-be. A detekciós felbontás és képsebesség azonban szándékosan alacsony – a jelen rendszerben 640×360 pixel és 5 kép/másodperc –, mert a neurális háló amúgy is leskálázott képen dolgozik. Az élő nézet így a rögzített felvételnél lényegesen rosszabb minőségű képet mutatott, holott a telefon 1280×720 pixeles folyamatot szolgáltatott. A második, hogy a JPEG-kódolás nézőnként külön processzorterhelést jelent: minden megnyitott kamerakártya egy-egy JPEG-enkódert indított a Frigate-ben. A harmadik, hogy az élő nézet minősége és a detekció beállításai összeesetelődtek: a detekciós felbontás emelése az élőkép javításáért fölösleges detektorterhelést okozott volna.

A megoldás a go2rtc MSE-csatornájának használata. A go2rtc – amely a Frigate képátlójába beágyazva fut – a telefon H.264 folyamatát fMP4 szegmensekké csomagolja át, *újrakódolás nélkül*, és WebSocket-en szolgáltatja. A backend ezt a folyamatot játssza át a böngészőnek, ahol a Media Source Extensions API-n [15] keresztül a natív H.264 dekóder jeleníti meg. A relay implementációja szándékosan minimális: két, egymással szemben futó továbbító feladat, amelyek közül az egyik befejeződése lezárja a másikat is.

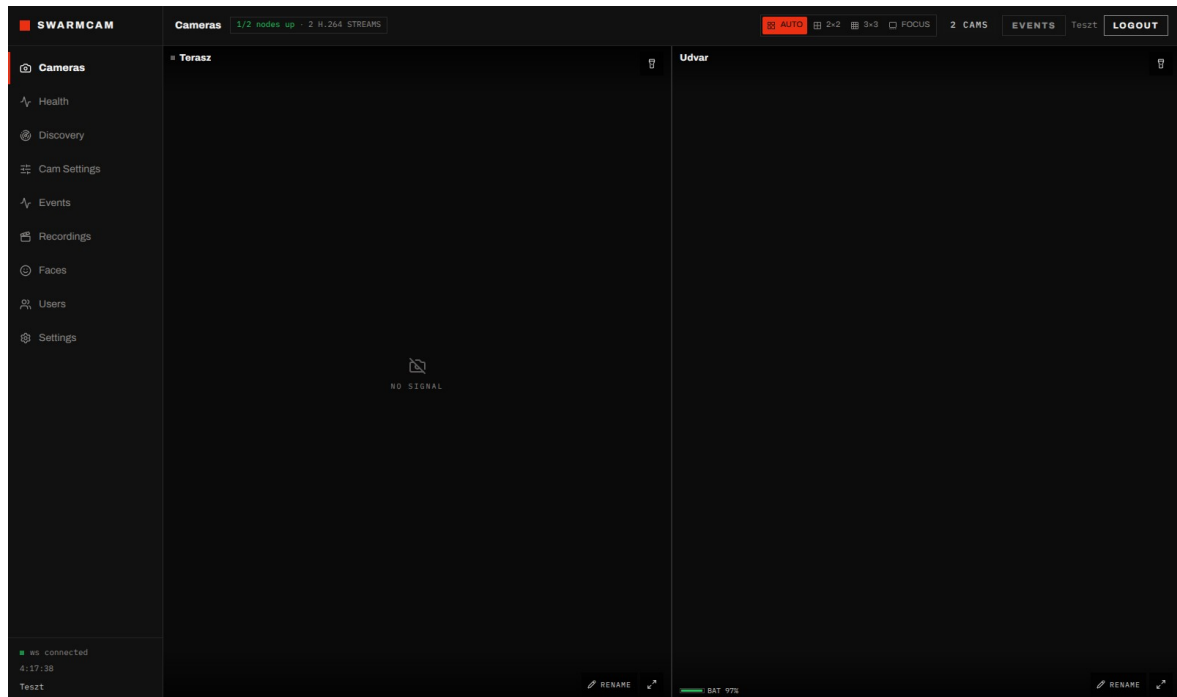
```
@router.websocket("/ws/cameras/{name}/mse")
async def camera_mse(ws: WebSocket, name: str):
    await ws.accept()
    async with websockets.connect(f"{GO2RTC_WS_URL}?src={quote(name)}",
                                max_size=None) as up:
        async def go2rtc_to_client() -> None:
            async for msg in up:
                if isinstance(msg, bytes):
                    await ws.send_bytes(msg)
                else:
                    await ws.send_text(msg)
```

A választás mellett szóló érvet a mérés adta: a váltás után az élő nézet a telefon natív felbontásán fut, a mért végponttól végpontig terjedő késleltetés 0,03–0,07 másodperc, és a Frigate processzorterhelése nézőnként körülbelül két százalékponttal nő a korábbi huszonhat helyett. A számszerű összehasonlítást a 3.6.5. szakasz közli.

Indokolt kitérni arra, hogy miért nem WebRTC lett a választás, holott a go2rtc azt is támogatja. A WebRTC ICE-tárgyalást és UDP-porthasználatot igényel, ami a fordított proxy és a hitelesítési útvonal konfigurációját is érinti. Az MSE ezzel szemben ugyanazon a WebSocket-en halad, amely már átmegy a meglévő nginx- és hitelesítési láncon, ugyanazt a H.264 továbbítást végzi újrakódolás nélkül, és a helyi hálózaton nem mérhető késleltetéstöbbletet okoz. A WebRTC előnyei – NAT-átjárás, adaptív bitráta – a helyi hálózaton értelmüket veszítik.

A régi MJPEG-proxy nem került eltávolításra: tartalékútvonalként megmaradt, és három sikertelen MSE-csatlakozási kísérlet után automatikusan átveszi a helyét, továbbá a fókusz nézet alsó sávjában látható előnézeti képeket is ez szolgálja ki. Ez a döntés a hibatűrési

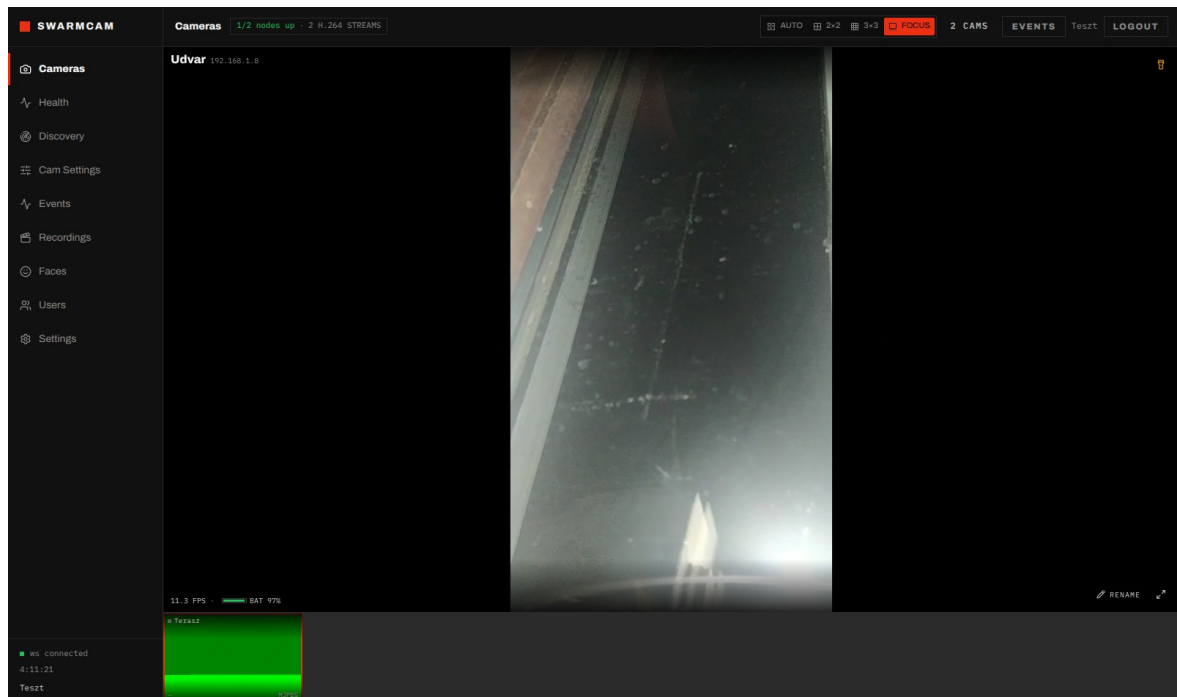
követelményből következik: ha a go2rtc-hez vezető út bármely okból nem járható, a felhasználó akkor is lát képet.



3. ábra: A dashboard Cameras nézete két regisztrált node-dal. A fejlécben a „1/2 nodes up” jelző mutatja, hogy az egyik telefon éppen nem elérhető; a hozzá tartozó kártya NO SIGNAL állapotot jelez, miközben a másik node élőképe és állapotadatai zavartalanul frissülnek.

A 3. ábra a rendszer normál üzemi nézetét mutatja abban a – valós üzemben rendszeresen előforduló – helyzetben, amikor az egyik node nem érhető el. Ez jól szemlélteti a hibatűrési követelmény teljesülését: egyetlen node kiesése nem befolyásolja a többi működését, és a felület a kiesést egyértelműen jelzi ahelyett, hogy hibaüzenettel vagy üres képernyővel válaszolna.

A kamerarács négyféle elrendezésben használható: automatikus, kétszer kettes, háromszor hármas és fókusz. Az utolsó (4. ábra) egyetlen kamerát nagyít fel, a többi pedig az alsó sávban, kis előnézeti képként jeleníti meg – ez a nézet a folyamatos megfigyelés tipikus munkamódja. Az előnézeti képeket a 3.5.3. szakasz végén említett MJPEG-útvonal szolgálja ki, mert egyszerre több MSE-folyam futtatása fölöslegesen terhelné a böngészőt.



4. ábra: Fókusz elrendezés. A kiválasztott kamera a teljes területet elfoglalja, a többi node az alsó sávban, előnézeti képként marad követhető; a kiemelt kamera alatt a valós, mért képsebesség és az akkumulátorszint látható.

Egyetlen kamera teljes képernyős megjelenítése külön nézetben is elérhető (5. ábra). Ez a nézet a háttérben futó összes többi élő folyamat szünetelteti – amit a fejléc külön jelez is –, így a megjelenített kamera a rendelkezésre álló sávszélességet és dekódolási kapacitást egyedül használja.



5. ábra: Teljes képernyős kameranézet. A fejléc jelzi, hogy a háttérben futó élő folyamatok szünetelnek; a lábléc a képsebességet, az akkumulátorszintet és a Wi-Fi jelerősséget mutatja.

3.5.4. A kameraforgatás megvalósítása

A telefonok elhelyezése ritkán teszi lehetővé, hogy a kép eleve a kívánt tájolásban érkezzen. Kézenfekvő megoldásnak tűnne a telefon saját forgatási beállításának használata, ez azonban a gyakorlatban nem működik: az alkalmazás forgatási beállítása a H.264 folyamra nincs hatással. A képkocka mérete és tájolása változatlan marad; az alkalmazás csupán lekicsinyíti a képet, és fekete sávok közé illeszti bele. Az eredmény felbontásvesztés, forgatás nélkül.

A forgatás ezért a go2rtc feldolgozási láncába került. Forgatás nélkül a go2rtc a telefon folyamatát változatlanul veszi át – nincs átkódolás. Forgatás esetén a lánc két tagúvá válik: egy nyers folyam veszi át a telefon képét, egy második pedig ebből képez elforgatott változatot. A forgatás így egyetlen helyen történik, és minden fogyasztó – a detekció, a felvétel, a pillanatkép és az élőkép – ugyanazt az elforgatott képet kapja.

Két részlet érdemel említést. A forgatáshoz szoftveres átkódolás szükséges – a hardveres gyorsítás ezen az útvonalon nem forgat, hanem a forrást a célméretre skálázza és feketével tölti ki a maradékot –, ezért a forgatott kamerák szoftveres kódolóval dolgoznak, de csak azok, amelyeknél erre ténylegesen szükség van. A forgatás állapotát pedig nem külön állomány, hanem maga a Frigate konfigurációjának forrásleírója tárolja, így nincs két, egymástól elcsúszható igazságforrás.

Fontos korlát, hogy a forgatás nem javítja ki az álló tájolású telefont. Ha a készülék képernyőforgatása állóra van rögzítve, az alkalmazás álló képet illeszt bele a fekvő képkockába – a mérés szerint a képkocka 68 százaléka fekete sáv. Ezt a go2rtc-oldali forgatás nem tudja visszanyerni, mert a képpontok már nem léteznek. A megoldás a telefon oldalán van: a képernyőforgatás feloldásával az alkalmazás fekvő képet ad, és a kamera forgatás – tehát átkódolás – nélkül használható.

3.5.5. A backend és a valós idejű kommunikáció

A backend két háttérfolyamatot futtat, amelyek a rendszer valós idejű viselkedéséért felelnek. Mindkettő a FastAPI indulásakor jön létre, és a folyamat teljes életciklusa alatt fut.

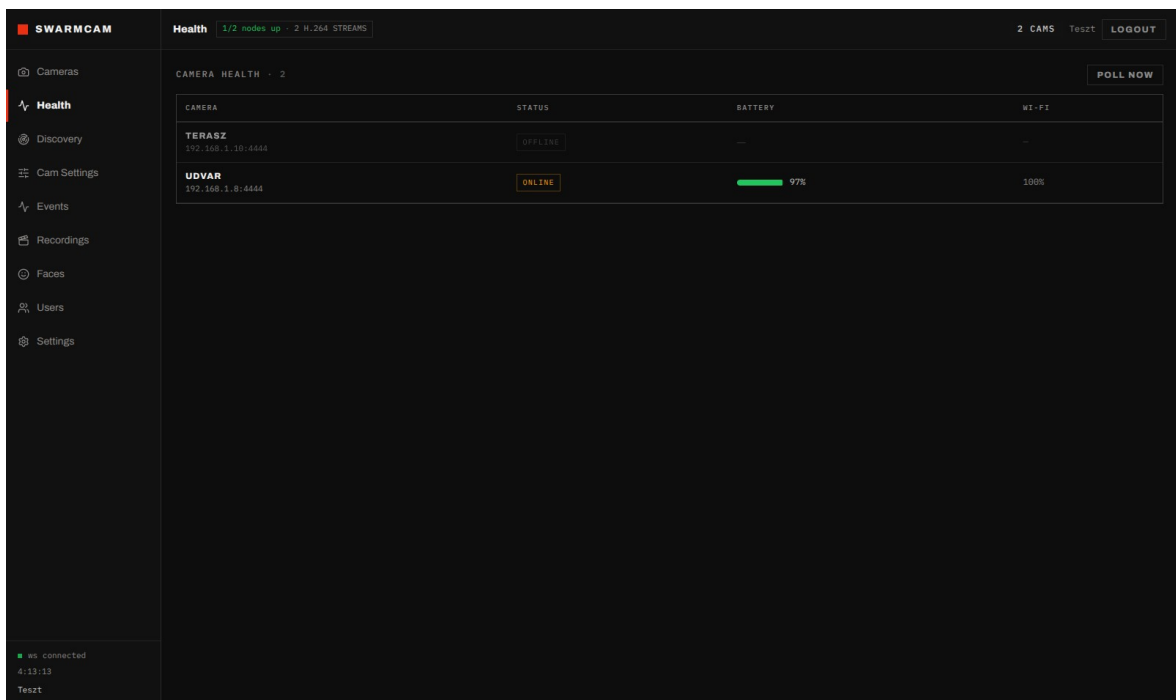
Az első az állapotfigyelő hurok. Ötmásodpercenként végigmegy a nyilvántartott kamerákon, mindegyiktől lekéri a /info.json leíró két másodperces időkorláttal, majd az eredményt a memóriabeli állapot-gyorsítótárba írja, és a teljes frissítést egyetlen üzenetben elküldi minden kapcsolódott WebSocket-kliensnek. A lekérés blokkoló HTTP-hívás, ezért végrehajtószálra kerül, hogy ne blokkolja az eseményhurkot.

```
async def health_loop() -> None:
    while True:
        updates = []
        for cam in state._last_cameras:
            status = await poll_status(cam["ip"], cam["port"])
            entry = {"ip": cam["ip"], "name": cam["name"],
                    "online": status is not None}
            if status:
                entry.update(_parse_info(status))
```

```
state._health_cache[cam["ip"]] = entry
updates.append(entry)
if state._ws_clients and updates:
    msg = json.dumps({"type": "status", "cameras": updates})
    await broadcast(msg)
await asyncio.sleep(HEALTH_INTERVAL)
```

Az öt másodperces periódus kompromisszum. Rövidebb periódus gyorsabb hibafelismerést adna, de a telefonokat feleslegesen terhelné – minden lekérés ébren tartja a rádiót és fogyasztja az akkumulátort. Hosszabb periódus mellett egy kiesett node felismerése észrevehetően lassulna. A választott érték mellett egy node kiesése legkésőbb hét másodperccel az esemény után megjelenik a felületen, ami a beállított két másodperces időkorlát és az öt másodperces periódus összege.

Az állapotadatok kiolvasása nem triviális leképezés. Az aktív kameraszensor azonosítója a leíróban hol tájolási kulcsszó („front”, „back”), hol pedig a szenzorlista egy elemének azonosítója („0”, „1”, „1:3”) – a kód mindkét esetet kezeli. A tájolás pedig kifejezetten a felhasználó által beállított forgatásból származik, nem a szenzor fizikai beépítési szögéből, mert utóbbi állandó, és a megjelenítés szempontjából érdektelen.



The screenshot shows the 'Health' section of the SWARMCAM web interface. It features a sidebar with navigation options like Cameras, Health, Discovery, Cam Settings, Events, Recordings, Faces, Users, and Settings. The main content area displays 'CAMERA HEALTH - 2' with a 'POLL NOW' button. Below this is a table with columns for CAMERA, STATUS, BATTERY, and WI-FI. Two cameras are listed: TERASZ (offline) and UDVAR (online with 97% battery). A status bar at the bottom indicates '2 ws connected' and the time '4:13:13'.

CAMERA	STATUS	BATTERY	WI-FI
TERASZ 192.168.1.10:4444	OFFLINE	—	—
UDVAR 192.168.1.8:4444	ONLINE	97%	100%

6. ábra: A Health nézet. Az állapotfigyelő hurok által gyűjtött adatok node-onként: elérhetőség, akkumulátorszint és Wi-Fi jelerősség. A táblázat a WebSocket-en érkező frissítésekkel valós időben változik.

A 6. ábra a rendszer állapotfigyelő nézetét mutatja. A táblázat nem lekérdezésre frissül: az adatok az állapotfigyelő hurok WebSocket-üzeneteiből érkeznek, tehát a felület a böngésző beavatkozása nélkül követi a valóságot.

A második háttérfolyamat az MQTT-hurok. Egyetlen tartós kapcsolatot tart fenn a brókerrel, és két témára iratkozik fel: a Frigate objektumdetekciós eseményeire és a tűzdetektor találataira. A két eseménytípust eltérően kezeli. A Frigate eseményét egyszerűen átalakítja a

felület által várt alakra, és továbbítja a WebSocket-klienseknek. A tűzeseményt ezzel szemben előbb egy különálló SQLite adatbázisba menti, az esemény mellé érkező annotált képkockát pedig a fájlrendszerre írja, és csak ezután küldi tovább – két üzenetben, egy felugró riasztásként és egy eseménylista-bejegyzésként.

Indokolt megválaszolni, miért kap a tűzesemény önálló adatbázist a Frigate saját tárolója helyett. A Frigate adatbázisa a Frigate belső implementációs részlete: sémáját a fejlesztők verzióról verzióra szabadon módosíthatják, és a rendszer semmilyen dokumentált módot nem kínál külső események beszúrására. Egy közvetlen írás a Frigate adatbázisába tehát olyan függőséget hozna létre egy nem publikus szerződéstől, amely bármelyik Frigate-frissítéskor eltörhet – ráadásul kétirányú írási versenyhelyzetet teremtene egy futó folyamattal. Ehhez képest egy néhány oszlopos, önálló SQLite tábla nulla üzemeltetési költséggel jár, teljes ellenőrzést ad a séma felett, és tisztán elválasztja a rendszer saját eseményforrását az integrált NVR-étől. Az elválasztás annak is feltétele, hogy a tűzdetektor a Frigate-től függetlenül, akár annak leállása mellett is tovább működhessen.

3.5.6. Tűz- és füstérzékelés

A feladatkiírás előírja a rendszer kiegészítését egy olyan eseményosztály felismerésével, amelyet az alapul szolgáló NVR általános célú objektumdetektora nem támogat. A választott osztály a tűz és a füst; ezek felismerése biztonsági rendszerben nyilvánvaló gyakorlati értékkel bír, ugyanakkor a Frigate alapmodelljének címkekészletében – amely a COCO adathalmaz nyolcvan mindennapi objektumosztályára épül – nem szerepel.

A megoldás önálló mikroszolgáltatás, nem pedig a Frigate modelljének cseréje. Ennek oka, hogy a Frigate egyszerre egy modellt futtat: a tűzfelismerő modell betöltése az általános objektumdetekció – a személy-, jármű- és állatfelismerés – elvesztésével járna. A két feladat két külön modellt kíván, és a legegyszerűbb mód a párhuzamos futtatásukra, ha külön folyamatban élnek.

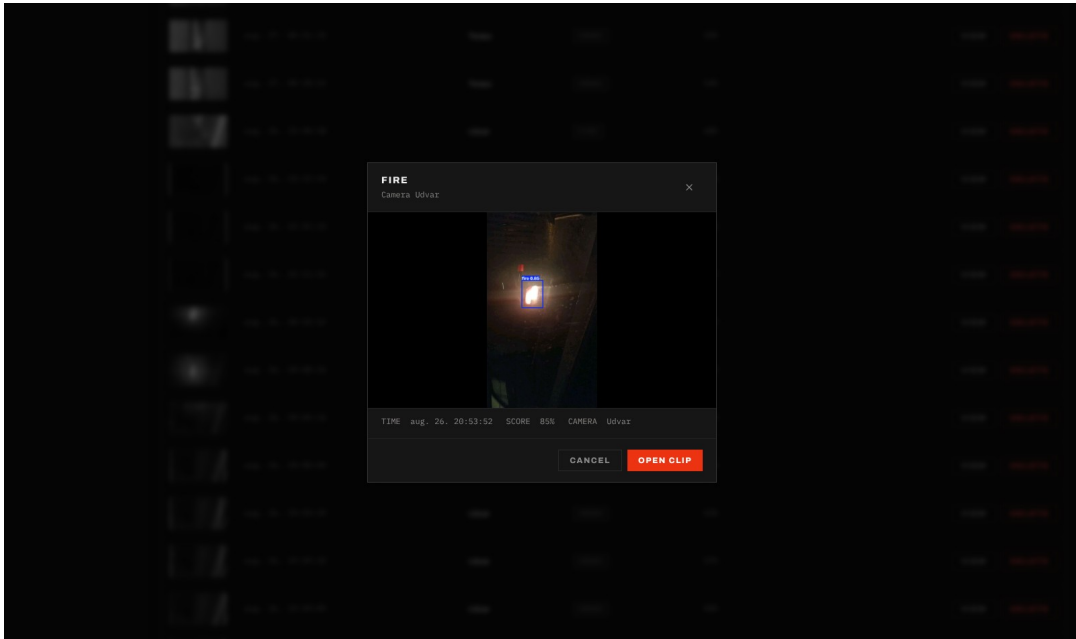
A szolgáltatás működése egyszerű ciklus. A backendtől lekéri az aktuális kameralistát, majd minden kameráról mintát vesz. A mintavétel a telefon MJPEG-folyamából történik: a szolgáltatás megnyitja a folyamatot, és az első teljes JPEG-képkockát olvassa ki belőle a JPEG-formátum kezdő- és zárójelzőinek megkeresésével, majd a kapcsolatot azonnal bontja. Erre azért van szükség, mert az alkalmazás nem kínál külön pillanatkép-végpontot.

A kivágott képkockán YOLO-modell fut. A modell egy nyilvánosan elérhető, tűz és füst osztályokra betanított súlykészlet [10], amely három osztályt ismer: tűz, füst és egyéb. A szolgáltatás csak az első kettőt veszi figyelembe, és csak akkor publikál, ha a modell magabiztossága eléri az ötven százalékos küszöböt.

```
results = model(img, verbose=False)
for r in results:
    for box in r.bboxes:
        conf = box.conf.item()
        if conf >= CONFIDENCE:
            label = model.names[int(box.cls.item())]
```

```
if label.lower() in ["fire", "smoke"]:  
    payload = {"camera": cam_name, "label": label.lower(),  
              "score": round(conf, 3)}  
    mqtt_c.publish(f"swarmcam/fire/{cam_name}",  
                  json.dumps(payload))
```

A találat mellé a szolgáltatás annotált – határolókeretekkel és magabiztossági értékkel ellátott – képkockát is generál, azt legfeljebb 1280 pixel szélességre kicsinyíti, JPEG-be kódolja, és base64 formában az üzenet részeként küldi. Így az eseményhez tartozó vizuális bizonyíték már a felület első megjelenítésekor rendelkezésre áll, külön lekérés nélkül.



7. ábra: Valós tűzdetekció eseményrészletei. A tűzdetekciós szolgáltatás 85 százalékos magabiztossággal azonosította a lángot; a képen a modell által rajzolt határolókeret és a hozzá tartozó pontszám látható.

A 7. ábra egy valós, éles környezetben rögzített tűzdetekciót mutat. A találat annotált képkockája a modell által rajzolt határolókeretet és a pontszámot is tartalmazza, így az esemény utólagos ellenőrzése nem igényel külön eszközt.

Licencjogi szempontból lényeges, hogy az inferenciát végző Ultralytics keretrendszer AGPL-3.0 licenc alatt áll. Ez a jelen, oktatási célú és nem terjesztett rendszer esetében nem jelent akadályt, kereskedelmi felhasználás esetén azonban vagy licencvásárlást, vagy a keretrendszer cseréjét igényelné – például a betanított súlyok ONNX formátumba exportálásával és ONNX Runtime alatti futtatásával, ami a modellt magát nem érinti.

A gyakorlati tapasztalat vegyes: a modell nappali fényviszonyok mellett valódi lángra és sűrű füstre magas magabiztossággal reagál, éjszakai, erősen zajos képeken viszont hajlamos téves füstdetekcióra. A jelenséget és annak okát a 3.6.7. szakasz tárgyalja.

3.5.7. A kezelőfelület

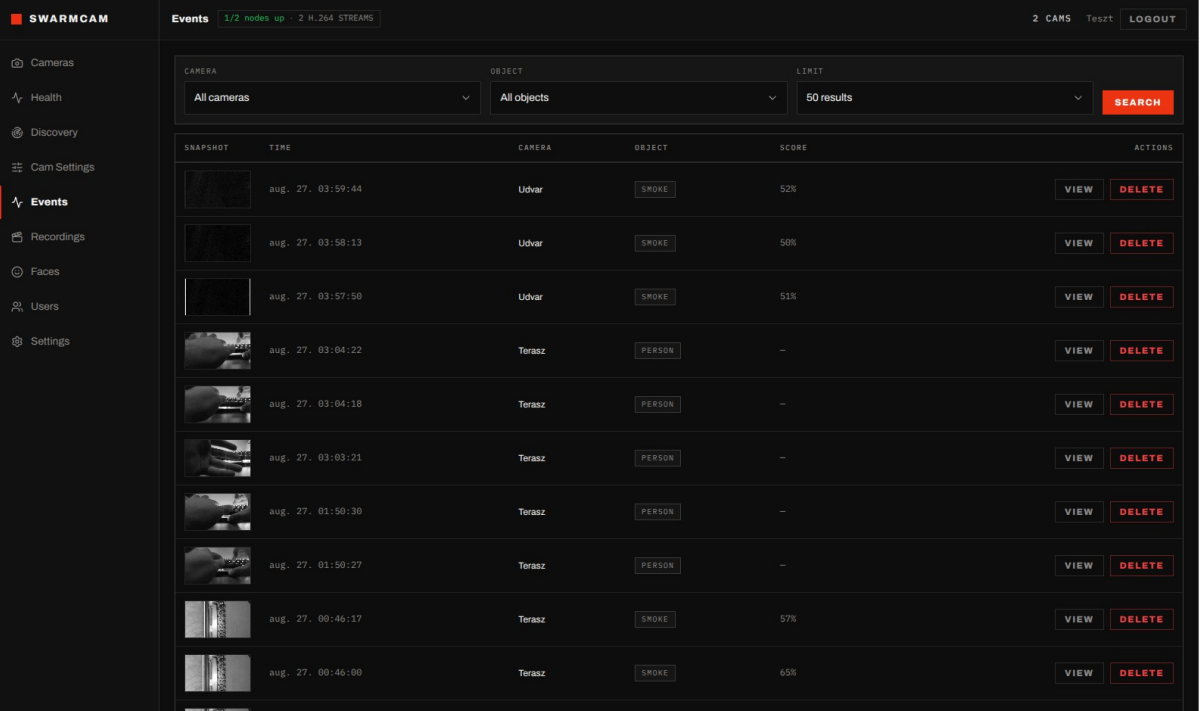
A kezelőfelület React és TypeScript alapú, Vite eszközlánccal épített egyoldalas alkalmazás; a megjelenítést Tailwind CSS és shadcn/ui komponensek adják. Az alkalmazás nem használ

útvonalválasztó könyvtárat: az aktív nézetet egyetlen állapotváltozó tárolja. Ez tudatos egyszerűsítés – a dashboard helyi adminisztrációs felület, amelyhez nem tartozik megosztható hivatkozás, tehát az útvonalválasztás mind a csomagméretet, mind a kód összetettségét fölöslegesen növelné.

A valós idejű frissítést egyetlen, központi kapcsolatkezelő biztosít. Ez tartja fenn a WebSocket-kapcsolatot, és a beérkező üzeneteket típus szerint osztja szét: az állapotüzenetek a kamerakártyák és a Health nézet adatait frissítik, az eseményüzenetek az eseményfolyamba kerülnek, a riasztások pedig felugró értesítésként jelennek meg. A kapcsolat bontását a hook automatikusan kezeli: öt másodperc múlva újracsatlakozik, kivéve ha a bontást maga a komponens életciklusa okozta.

```
socket.onclose = () => {  
  if (closedByCleanup) return  
  setStatus('reconnecting')  
  reconnectTimer = window.setTimeout(connect, 5000)  
}
```

Az események két helyen érhetők el. Az élő eseményfolyam a kamerarács mellé nyitható oldalpanel, amely a most érkező detektálásokat mutatja. A történeti eseménybrowser ezzel szemben a Frigate eseményadatbázisát kérdezi le a backend proxyján keresztül, kamerára, objektumtípusra és darabszámbra szűrhetően.



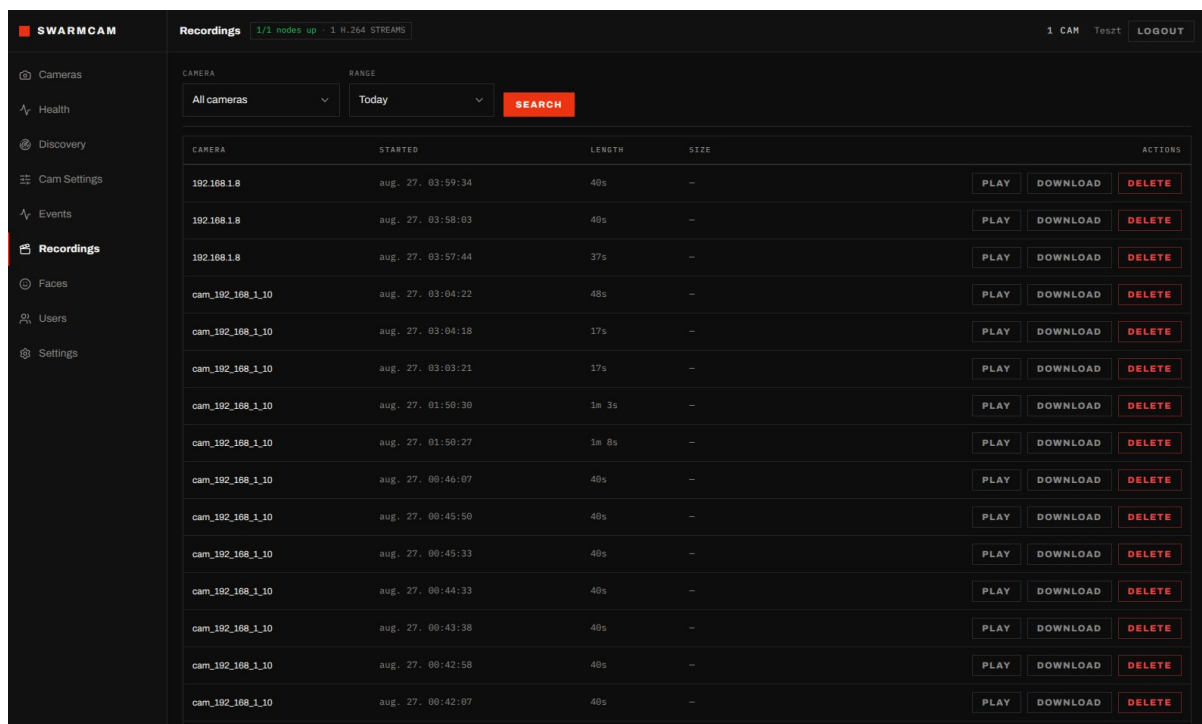
SNAPSHOT	TIME	CAMERA	OBJECT	SCORE	ACTIONS
	aug. 27. 03:59:44	Udvar	SMOKE	52%	VIEW DELETE
	aug. 27. 03:58:13	Udvar	SMOKE	50%	VIEW DELETE
	aug. 27. 03:57:50	Udvar	SMOKE	51%	VIEW DELETE
	aug. 27. 03:04:22	Terasz	PERSON	–	VIEW DELETE
	aug. 27. 03:04:18	Terasz	PERSON	–	VIEW DELETE
	aug. 27. 03:03:21	Terasz	PERSON	–	VIEW DELETE
	aug. 27. 01:50:30	Terasz	PERSON	–	VIEW DELETE
	aug. 27. 01:50:27	Terasz	PERSON	–	VIEW DELETE
	aug. 27. 00:46:17	Terasz	SMOKE	57%	VIEW DELETE
	aug. 27. 00:46:00	Terasz	SMOKE	65%	VIEW DELETE

8. ábra: Az események böngészője. A találatok között a Frigate objektumdetekciói (person) és a tüzdetekciós mikroszolgáltatás találatai (smoke, fire) egységes listában jelennek meg, kamerára és objektumtípusra szűrhetően.

A 8. ábra jól szemlélteti a két eseményforrás egyesítését: a Frigate személydetektálásai és a saját tüzdetektor füst- és tűztalálatai ugyanabban a listában, azonos szerkezettel jelennek

meg, holott két, egymást nem ismerő komponenstől származnak. A felhasználó számára a forrás különbsége nem látszik – ezt teszi lehetővé az MQTT-alapú szétcsatolás.

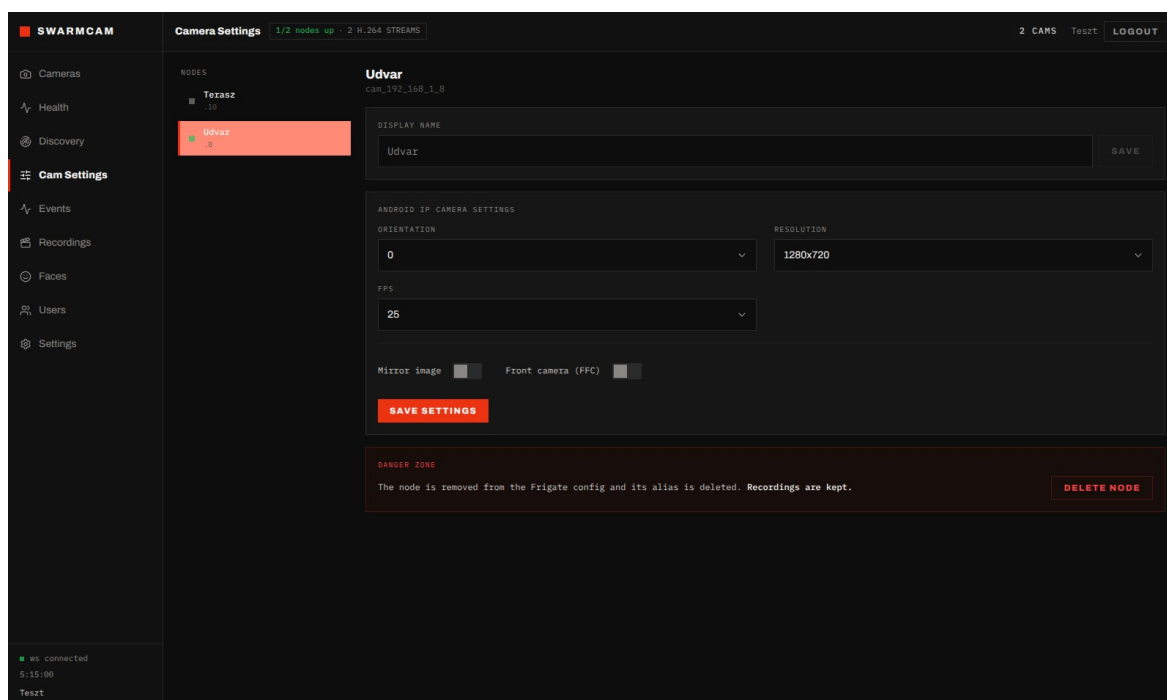
A felvételek külön nézetben böngészhetők, kamerára és időtartományra szűrve (9. ábra); a lejátszás és a letöltés is a backend proxyján halad, tehát a böngésző soha nem lép közvetlen kapcsolatba a Frigate-tel.



CAMERA	STARTED	LENGTH	SIZE	ACTIONS
192.168.1.8	aug. 27. 03:59:34	40s	–	PLAY DOWNLOAD DELETE
192.168.1.8	aug. 27. 03:58:03	40s	–	PLAY DOWNLOAD DELETE
192.168.1.8	aug. 27. 03:57:44	37s	–	PLAY DOWNLOAD DELETE
cam_192_168_1_10	aug. 27. 03:04:22	48s	–	PLAY DOWNLOAD DELETE
cam_192_168_1_10	aug. 27. 03:04:18	17s	–	PLAY DOWNLOAD DELETE
cam_192_168_1_10	aug. 27. 03:03:21	17s	–	PLAY DOWNLOAD DELETE
cam_192_168_1_10	aug. 27. 01:50:30	1m 3s	–	PLAY DOWNLOAD DELETE
cam_192_168_1_10	aug. 27. 01:50:27	1m 8s	–	PLAY DOWNLOAD DELETE
cam_192_168_1_10	aug. 27. 00:46:07	40s	–	PLAY DOWNLOAD DELETE
cam_192_168_1_10	aug. 27. 00:45:50	40s	–	PLAY DOWNLOAD DELETE
cam_192_168_1_10	aug. 27. 00:45:33	40s	–	PLAY DOWNLOAD DELETE
cam_192_168_1_10	aug. 27. 00:44:33	40s	–	PLAY DOWNLOAD DELETE
cam_192_168_1_10	aug. 27. 00:43:38	40s	–	PLAY DOWNLOAD DELETE
cam_192_168_1_10	aug. 27. 00:42:58	40s	–	PLAY DOWNLOAD DELETE
cam_192_168_1_10	aug. 27. 00:42:07	40s	–	PLAY DOWNLOAD DELETE

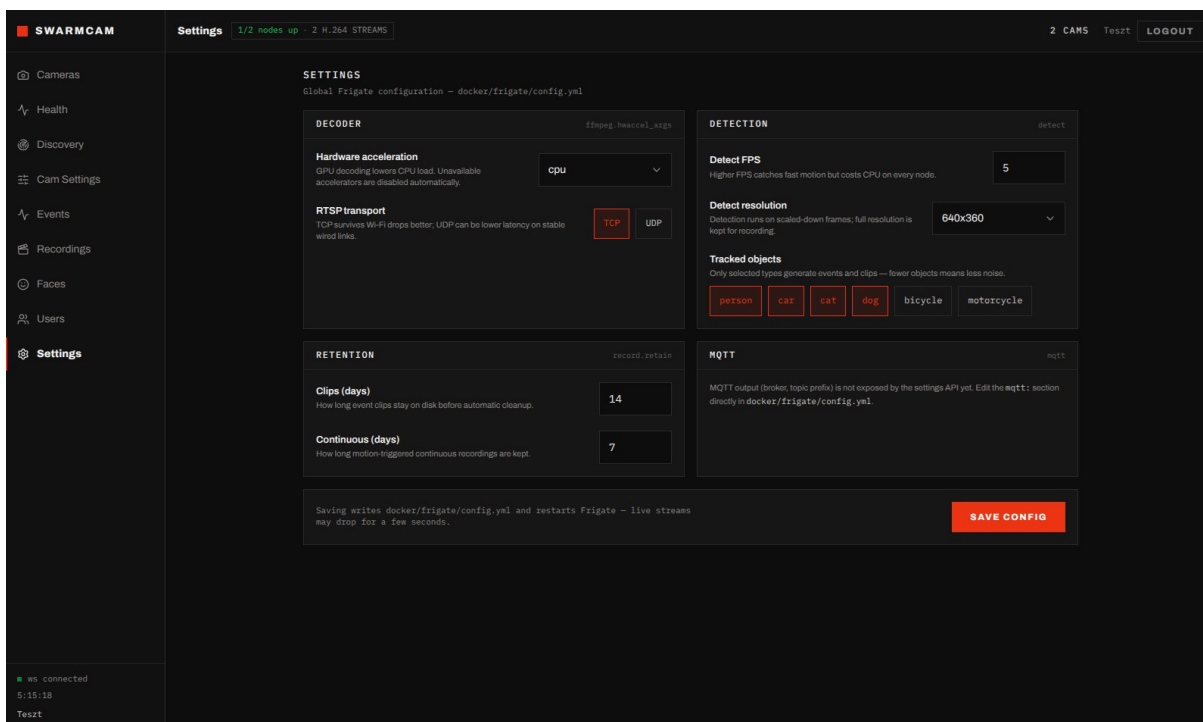
9. ábra: A felvételek böngészője. A klipek kameránként és időtartományonként szűrhetők, közvetlenül lejátszhatók és letölthetők.

A kamerabeállítások nézet (10. ábra) a kiválasztott node paramétereit módosítja: a megjelenített nevet, a tájolást, a felbontást, a képsebességet, a képtükrözést és az aktív kameraszenzort. A beállítások egy része a telefonra megy – egyetlen, több paramétert tartalmazó kérdésben –, a tájolás viszont a Frigate konfigurációjába kerül, a 3.5.4. szakaszban leírt okból.



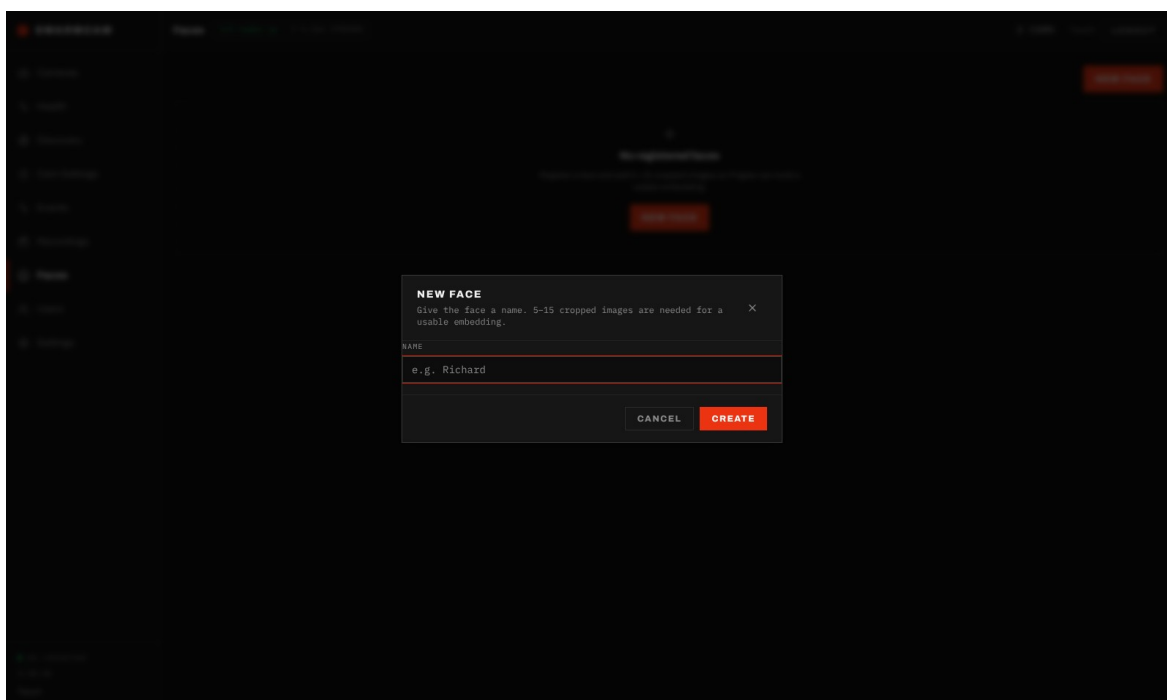
10. ábra: Kamerabeállítások. A bal oldali listából kiválasztott node paramétereit élőben módosíthatók; a „Danger zone” szekció a node eltávolítását teszi lehetővé, a hozzá tartozó felvételek megtartása mellett.

A rendszerbeállítások nézet (11. ábra) a Frigate globális konfigurációjának azon részeit teszi elérhetővé, amelyek a mindennapi üzemeltetés során valóban változhatnak: a dekóder gyorsítási módját, az RTSP-átvitel protokollját, a detekciós képsebességet és felbontást, a követett objektumtípusokat, valamint a felvételek megőrzési idejét. A mentés a konfigurációs állományt írja, majd újraindítja a Frigate konténert – a felület erre külön figyelmeztet is, mert az újraindítás alatt az élő folyamatok néhány másodpercre megszakadnak.



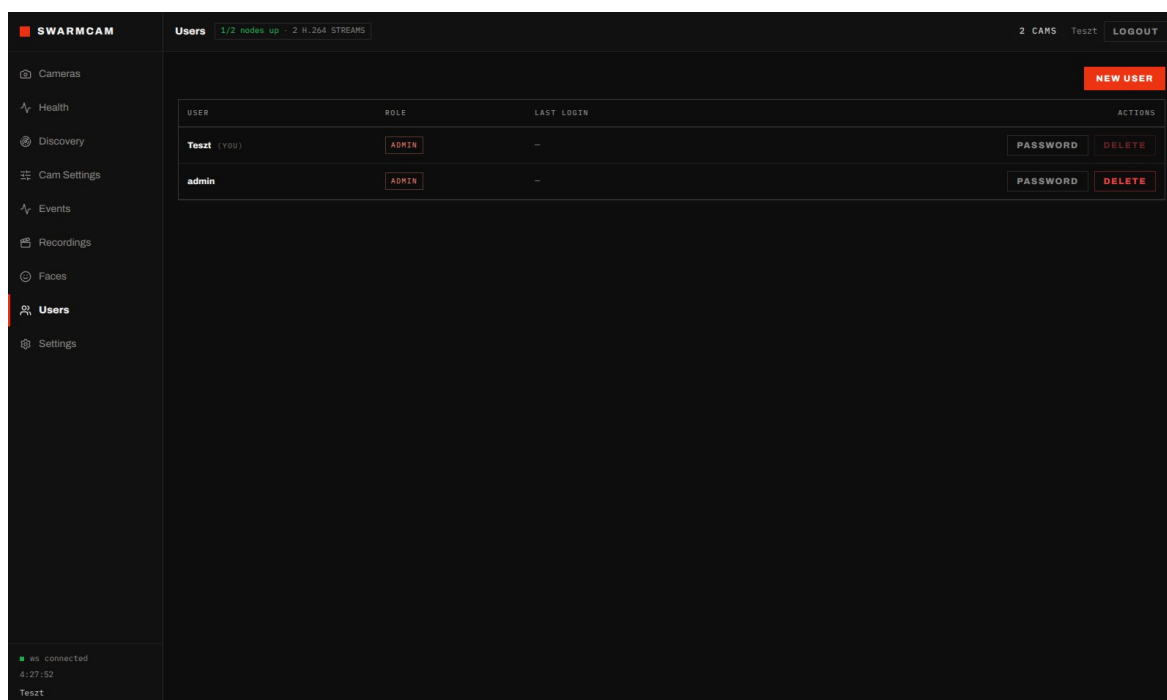
11. ábra: Rendszerbeállítások. A felület a Frigate konfigurációjának üzemeltetés közben ténylegesen változó részeit teszi elérhetővé; a mentés a konfigurációs állomány írása után újraindítja a Frigate konténert.

Az arcfelismerés kezelőfelülete (12. ábra) nem önálló SwarmCam-komponens, hanem a Frigate beépített képességének proxyzott felülete. Ez tudatos döntés: a Frigate az arcfelismerést ugyanabba a detekciós csővezetékbe illeszti, amelybe az objektumdetekció is, és a felismert arcokat közvetlenül az eseményekhez rendeli. Egy külön arcfelismerő szolgáltatás ezt az összekapcsolást elvesztené.



12. ábra: Új arc regisztrálása. A felület a Frigate arcfelismerő API-jának proxyzásán keresztül működik; a megbízható felismeréshez a Frigate 5–15 kivágott arcképet igényel.

A felhasználókezelés (13. ábra) hasonló elven működik: a Frigate saját felhasználói adatbázisát kezeli a backend proxyján keresztül. A rendszernek így nincs önálló felhasználónyilvántartása, tehát nincs két, egymástól elcsúsztatható jogosultsági modell sem.



13. ábra: Felhasználókezelés. A rendszer nem tart fenn saját felhasználónyilvántartást: a Frigate felhasználói adatbázisát kezeli a backend proxyján keresztül.

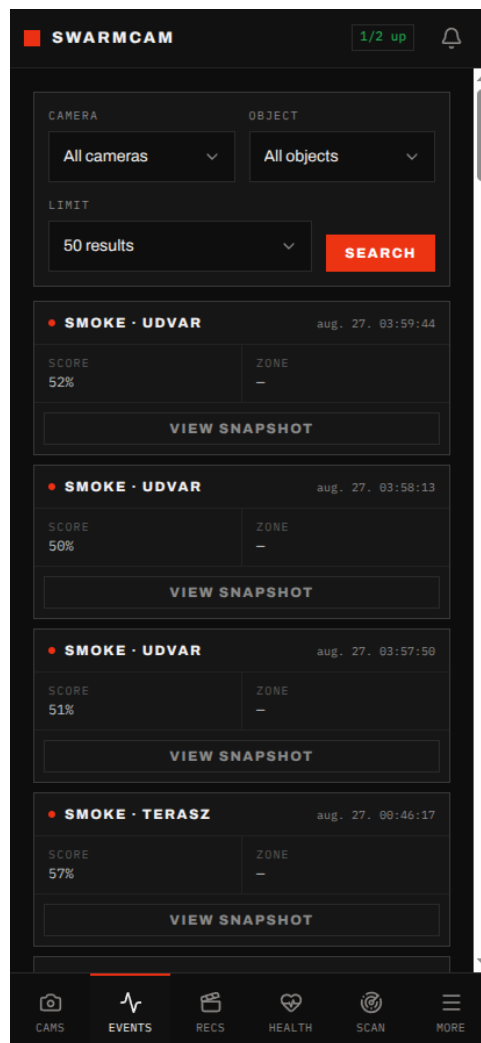
3.5.8. Mobilnézet

A felület reszponzív: a megjelenítés a rendelkezésre álló szélesség alapján két, egymástól határozottan eltérő elrendezés között vált. Ez nem csupán a rács oszlopszámának változtatása, hanem külön navigációs modell.

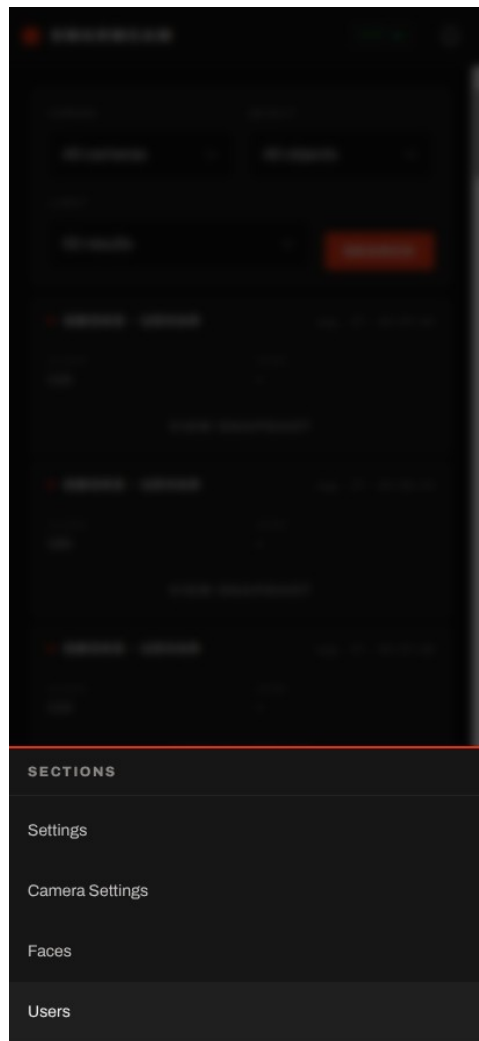
Asztali szélességen az oldalsáv állandóan látható, a nézetek között egy kattintással lehet váltani, és a fejlécben elfér a kamerarács elrendezésválasztója, az eseménypanel kapcsolója és a felhasználói menü. Mobil szélességen az oldalsáv eltűnik: helyette rögzített fejléc és alsó fülsáv jelenik meg, amely a hat leggyakrabban használt nézetet hüvelykujjal elérhető távolságban tartja; a többi nézet egy alulról felcsúszó lapon érhető el. A kamerarács elrendezésválasztója megmarad, így a felhasználó eldöntheti, hogy több kamerát lát egyszerre kisebb csempéken, vagy egyetlen kamerát a teljes szélességben.



14. ábra: A dashboard mobil elrendezése. Az oldalsáv helyét rögzített fejléc és a képernyő alján rögzített fűlsáv veszi át; a kamerarács a rendelkezésre álló szélességhez igazodik.



15. ábra: Az események böngészője mobil elrendezésben. Az asztali táblázat kártyás nézetű alakul, hogy vízszintes görgetés nélkül olvasható maradjon.



16. ábra: A „More” lap mobil nézetben. A fülsávba nem férő nézetek egy alulról felcsúszó lapon érhetők el.

A 14. ábra a kamerarácsot, a 15. ábra az eseménybrowsert mutatja mobil elrendezésben; utóbbin jól látszik, hogy az asztali táblázatból kártyalista lesz, mert egy hatoszlopos táblázat telefonon vízszintes görgetés nélkül olvashatatlan volna. A fülsávba nem férő nézetek a 16. ábrán látható, alulról felcsúszó lapon érhetők el.

A mobilnézet létjogosultsága gyakorlati: egy otthoni biztonsági rendszer tipikus használati módja az, hogy a felhasználó telefonon néz rá a kamerákra. Az élőkép ebben a nézetben ugyanazon az MSE-csatornán fut, mint az asztali változatban – a mobil böngészők natív H.264 dekódere ugyanúgy kezeli az fMP4 szegmenseket.

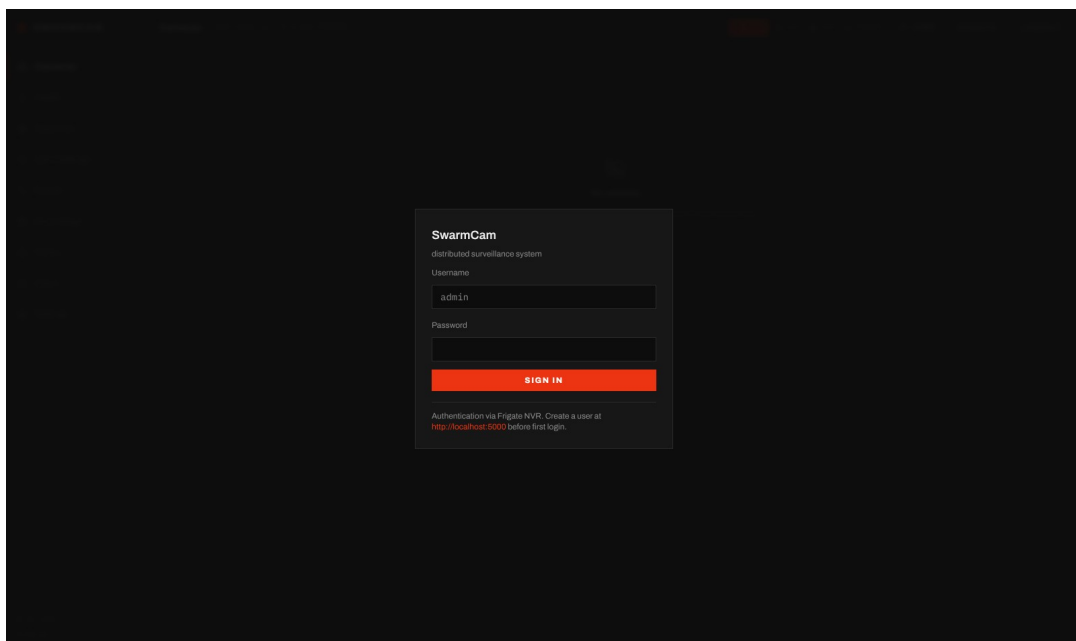
3.5.9. Hitelesítés és biztonsági megfontolások

A hitelesítés a Frigate felhasználói adatbázisára épül. A felhasználó a bejelentkezési űrlapon (17. ábra) megadott adatait a backend továbbítja a Frigate bejelentkezési végpontjának, majd a válaszul kapott JWT-t adja vissza a felületnek. Minden további kérés ezt a tokent hordozza, a backend pedig érvényesíti és továbbadja.

A frontend a tokent kizárólag a React alkalmazás memóriájában, kontextusban tárolja – nem a böngésző tartós tárolójában. Ez tudatos döntés: a tartós tárolóban elhelyezett token minden, az oldalon futó szkript számára olvasható, tehát egy sikeres szkriptbeillesztéses támadás azonnal átvehető munkamenetet ad. A memóriában tárolt token az oldal újratöltésekor elvész – a felhasználónak újra be kell jelentkeznie –, ami kényelmetlenség, de egy helyi adminisztrációs felületen elfogadható ár.

A backend ezen felül külön, saját hitelesített munkamenetet is fenntart a Frigate felé. Erre azért van szükség, mert az állapotfigyelés, a statisztikák lekérése és a konfiguráció olvasása felhasználói beavatkozás nélkül, folyamatosan zajlik – ezek nem köthetők egyetlen bejelentkezett felhasználó munkamenetéhez. A két munkamenet független: a felhasználói token lejárta nem érinti a háttérfolyamatokat, a háttérmunkamenet automatikus megújítása pedig nem befolyásolja a bejelentkezett felhasználót.

A hibakezelés két HTTP-státuszkódot különböztet meg élesen. A 401-es válasz – érvénytelen vagy lejárt token – kijelentkezteti a felhasználót. Az 503-as válasz – a Frigate átmenetileg nem elérhető, például éppen újraindul egy konfigurációmentés után – viszont nem: ilyenkor a felület hibaüzenetet mutat, de a munkamenetet megtartja. E két eset összemosása azt eredményezné, hogy minden konfigurációmentés kijelentkeztetné a felhasználót.



17. ábra: Bejelentkezési felület. A hitelesítés a Frigate felhasználói adatbázisára épül; a kapott JWT kizárólag az alkalmazás memóriájában él, tartós böngészőtárolóba nem kerül.

Végül fontos rögzíteni a rendszer biztonsági modelljének határát. A SwarmCam kizárólag megbízható, helyi hálózaton üzemelő rendszernek készült: az MQTT-bróker névtelen hozzáférést enged, a kamerák önálló tanúsítványát a rendszer ellenőrzés nélkül elfogadja, a konténerek nagy része a gazdagép hálózati névterét használja, és a backend hozzáfér a Docker vezérlőaljzatához. Ezek a döntések a helyi hálózat feltételezésével együtt védhetők;

internet felőli közvetlen kitettség esetén viszont mindegyik önálló sebezhetőséggé válna. Távoli elérésre a helyes megoldás a rendszer elé helyezett VPN, nem pedig portátírányítás.

3.6. Tesztelés és mérési eredmények

A rendszer valós otthoni hálózati környezetben, több különböző generációjú Android eszközzel került tesztelésre; a fejlesztés teljes ideje alatt éles, folyamatos üzemben működött. Ez a szakasz a védelem előtt elvégzett, számszerű mérések eredményeit közli, és mindegyik mérésnél megadja azt a módszertani részletet, amely nélkül az eredmény nem értelmezhető helyesen.

3.6.1. A mérési környezet és a módszertan

A méréseket a 7. táblázatban leírt gépen végeztem, a teljes rendszer szokásos, konténerizált konfigurációjában.

7. táblázat: A mérési környezet

Elem	Érték
Processzor	AMD Ryzen 9 5900HX, 8 mag / 16 szál
Memória	15 GiB
GPU	NVIDIA GeForce RTX 3060 Laptop, 6 GiB, 595.84 illesztőprogram
Operációs rendszer	Ubuntu 24.04.3 LTS
Konténerkörnyezet	Docker 28.5.2, Docker Compose v2
Frigate	0.17.2, stable-tensorrt képfájl
Detektor	ONNX Runtime, yolo-generic modell, 320×320 bemenet, COCO-80 címkekezelés
Backend	Python 3.12, FastAPI 0.141
Kamera-node	Android IP Camera, 1280×720, H.264, HTTPS

A mérések során felmerült egy módszertani nehézség, amelyet a dolgozat szintén rögzít. A kameraszám függvényében vizsgált erőforrás-használathoz és a detektor összehasonlító méréséhez minden mérési pontban azonos bemenetre van szükség, különben a mérési pontok közötti különbség nem a vizsgált változóból, hanem a kamera előtt éppen zajló események eltéréséből származna. Egy élő telefonkamera ezt a feltételt nem teljesíti: a képtartalom és – ahogy a 3.6.6. szakasz mutatja – a képsebesség is folyamatosan változik a megvilágítással.

A megoldás az volt, hogy a mérés bemenete egy korábban, ugyanezekről a telefonokról rögzített, valós H.264 felvétel: 1280×720 felbontású, 10 kép/másodperces, huszonnégy perc hosszúságú állomány, amelyet a Frigate minden mérési pontban ugyanúgy, N különálló kameraként dolgoz fel. Ez két okból előnyös. Egyrészt minden mérési pont pontosan azonos

bemenetet kap, tehát a különbségek kizárólag a vizsgált változóból – a kameraszámból, illetve a detektor eszközéből – erednek. Másrészt a bemenet kodekje, felbontása és képsebessége megegyezik azzal, amit a telefonok ténylegesen szolgáltatnak, tehát a mért terhelés a valós üzemre nézve reprezentatív.

A módszer egyetlen eltérése az éles üzemtől az, hogy ebben az elrendezésben a Frigate közvetlenül olvassa a felvételt, a go2rtc újracsomagoló lépése nélkül. A go2rtc ezen a ponton nem kódol újra, csak konténerformátumot vált, így a hozzájárulása kicsi és állandó; a mért értékek ennek megfelelően a go2rtc terhelését nem tartalmazzák.

A mérőszkript (benchmark/measure.py) minden mérési pontban hatvan másodpercen át, kétmásodpercenként mintavételezi a konténerek erőforrás-használatát, és a minták átlagát, maximumát és darabszámát rögzíti. A mérési pontok között a rendszer hatvan másodpercet stabilizálódik, hogy a Frigate újraindítása utáni tranziens ne torzítsa az eredményt.

3.6.2. Erőforrás-használat a kameraszám függvényében

Az első mérés azt vizsgálja, hogyan skálázódik a rendszer erőforrásigénye a bekötött kamerák számával. A mérés nulla kamerával indul – ez adja a rendszer alapterhelését, vagyis azt, amit a futó konténerek videófeldolgozás nélkül fogyasztanak –, majd egytől négy kameráig halad.

8. táblázat: A konténerek erőforrás-használata a kameraszám függvényében (60 s alatt, 2 s-onként vett minták átlaga)

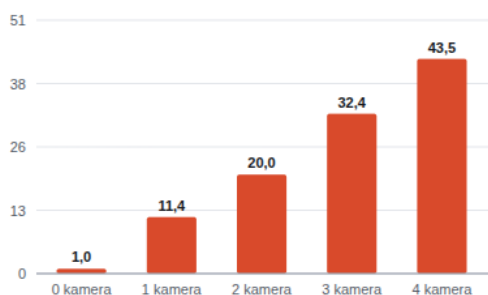
Kameraszám	0	1	2	3	4
frigate — CPU (%)	1,0	11,4	20,0	32,4	43,5
frigate — memória (MiB)	851	1414	1521	1632	1725
backend — CPU (%)	0,40	0,56	0,45	0,61	0,55
backend — memória (MiB)	48	49	50	52	52
fire-detector — CPU (%)	0,40	0,27	0,36	0,33	0,32
fire-detector — memória (MiB)	131	131	131	131	131

mosquitto — CPU (%)	0,04	0,04	0,03	0,03	0,03
dashboard — CPU (%)	0,00	0,01	0,01	0,01	0,01

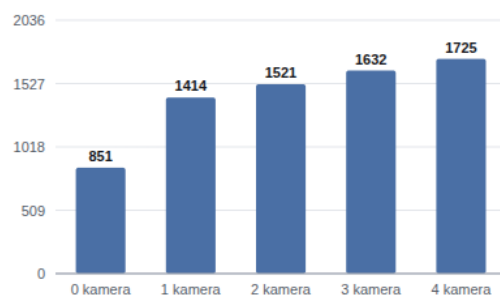
A mért értékeket a 8. táblázat, a Frigate konténer skálázódását pedig a 19. ábra foglalja össze. Az adatokból három következtetés vonható le.

Erőforrás-használat a kameraszám függvényében

FRIGATE — PROCESSZORHASZNÁLAT (%)



FRIGATE — MEMÓRIA (MiB)



19. ábra: A Frigate konténer processzor- és memóriahasználat a bekötött kamerák számának függvényében. A processzorterhelés kameránként közel állandó értékkel nő, a memóriaigény pedig az első kamera után gyakorlatilag telítődik.

Az első, hogy a rendszer alapterhelése elhanyagolható. Kamerák nélkül a Frigate processzorhasználat 1 százalék alatt marad (0,97%); ami mérhető, az szinte teljes egészében memória: a betöltött modell és a futtatókörnyezet önmagában is lefoglal 851 MiB-ot, függetlenül attól, hány kamera fut. Ez azt jelenti, hogy a rendszer folyamatos üzemben tartása kamerák nélkül gyakorlatilag ingyenes.

A második, hogy a processzorterhelés a kameraszámmal lényegében lineárisan nő: 1 kameránál 11,4%, kettőnél 20,0%, háromnál 32,4%, négyenél 43,5%. A növekmény kameránként 10,6 százalékpont körül állandó – a lineáris illesztés eltérése egyetlen mérési pontban sem haladja meg a két százalékpontot. Ez összhangban van azzal, amit a 2.3.3. szakaszban leírt csővezetékűtől várunk: minden kamera saját dekódolási folyamattal és saját mozgásmaszkkal fut, ezek költsége kameránként adódik össze, míg a detektor – amely a mérési bemeneten ritkán szólal meg – közös erőforrás. A memóriaigény ezzel szemben nem lineáris: az első kamera 563 MiB-ot ad hozzá, a további három együtt már csak 311 MiB-ot, mert a nagyobb, kameraszámtól független allokációk (modell, futtatókörnyezet, közös pufferek) az első kamera indulásakor egyszer jönnek létre.

A harmadik, hogy a többi konténer terhelése a kameraszámtól gyakorlatilag független. A backend processzorhasználat minden mérési pontban 1 százalék alatt marad, memóriaigénye 48–50 MiB között állandó; az üzenetbróker 3 MiB-ot és mérhetetlenül kevés processzoridőt, a webkiszolgáló 14 MiB-ot használ. A rendszer skálázódását tehát egyedül a

videófeldolgozó réteg határozza meg – a saját fejlesztésű komponensek nem jelentenek szűk keresztmetszetet.

A mért adatokból a gyakorlati üzemeltetésre nézve is levonható következtetés. A 43,5 százalékos érték a docker stats konvenciója szerint egyetlen processzormag terheléséhez viszonyítva értendő, tehát négy kamera a tizenhat szálás gép teljes kapacitásának alig 2,7 százalékát köti le. A lineáris skálázódásból extrapolálva a mérési gép elméletileg több tucat kamerát is kiszolgálna; a valós korlátot ennél jóval hamarabb a memória, a lemezes írási sávszélesség és – gyorsítás nélküli detektor esetén – a detektor átbocsátóképessége szabja meg.

3.6.3. Detektálási sebesség: GPU-gyorsítás és processzoros futtatás

A második mérés a hardveres gyorsítás hatását vizsgálja a detektálási sebességre. Az összehasonlítás tisztaságát az biztosítja, hogy a két futtatás mindenben megegyezik – azonos modell, azonos bemenet, azonos kameraszám, azonos detekciós felbontás –, és kizárólag a detektor eszköze tér el: az egyik esetben az ONNX Runtime a GPU-t használja, a másikon kifejezetten a processzorra van kényszerítve.

A mérés egy további módszertani beavatkozást igényelt. Az alapbeállítás szerinti mozgásmaszk mellett a detektor a mérési bemeneten túl ritkán szólal meg ahhoz, hogy a következtetési idő megbízhatóan mérhető legyen – ez éppen a csővezeték hatékonyságából következik. Ezért a két összehasonlító futtatás alatt a mozgásérzékelés küszöbe minimumra van állítva, ami a detektort gyakorlatilag minden képkockán megszólaltatja. Az így mért következtetési idő tehát telített detektort jellemez; a valós üzemben ugyanez a detektor sokkal ritkábban dolgozik.

9. táblázat: A detektor következtetési ideje GPU-gyorsítással és processzoron (azonos modell, azonos bemenet, 3 kamera, telített detektor)

Mutató	GPU (ONNX Runtime, CUDA)	Processzor (ONNX Runtime, CPU)
Következtetési idő képkockánként (ms)	5,54	43,12
Legnagyobb mért következtetési idő (ms)	5,57	47,07
Összesített detekciós képsebesség (kép/s)	15,68	14,07
Elméleti detekciós felső korlát (kép/s)	180,5	23,2

A 9. táblázat szerint a GPU-gyorsítás a képkockánkénti következtetési időt 43,12 ezredmásodpercről 5,54 ezredmásodpercre csökkenti – ez 7,8-szeres gyorsulás, azonos modellel és azonos bemenettel. Három értelmezési megjegyzés kívánczik mellé.

Az első, hogy a processzoros futtatás nem használhatatlan – csak a kiszolgálható kameraszámot korlátozza. A detektor egyetlen, sorosított erőforrás: ha egy képkocka feldolgozása n ezredmásodperc, akkor a rendszer másodpercenként legfeljebb $1000/n$ detektálást tud elvégezni, függetlenül attól, hány kamera kéri, és ez az érték oszlik szét a kamerák között. A mért értékekből ez processzoron 23,2, GPU-n 180,5 detektálás másodpercenként.

A második, hogy ezt a korlátot a mérés éppen elérte, de nem lépte túl. Három kamera, egyenként 5 kép/másodperces detekciós beállítással 15 detektálást igényel másodpercenként; a processzoros futtatás ebből 14,07-et teljesített, a GPU-s 15,68-at. A processzor tehát ezen a terhelésen még majdnem lépést tart – a 23,2-es elméleti korlát viszont azt is megmutatja, hogy körülbelül négy-öt kameránál telítődne, míg a GPU ugyanezen a beállításon több mint harminc kamerát is kiszolgálna.

A harmadik, hogy a mérési gép processzora tizenhat szál, tehát viszonylag erős. Egy tipikus, alacsony fogyasztású mini PC-n a processzoros érték ennél lényegesen rosszabb lenne, ami a GPU – vagy alternatívaként egy Coral típusú célhardver – melletti érvet tovább erősíti.

A mérés gyakorlati tanulsága, hogy a rendszer alapértelmezett konfigurációja szándékosan processzoros detektorral indul, hiszen az minden gépen működik, a GPU-gyorsítás pedig a beállítások felületén, egyetlen legördülő menüből kapcsolható be, ha a gazdagépen van támogatott gyorsító.

3.6.4. Eseménykésleltetés és az állapotfigyelés pontossága

A harmadik mérés a rendszer valós idejű viselkedését vizsgálja két mutatón keresztül. Az első az állapotfigyelő hurok periódusának tényleges szórása: az öt másodpercre tervezett ciklus mennyire tartja magát a névleges értékhez, amikor közben a kamerákat is lekérdezi. A második a rendszer saját eseményátjátszási késleltetése: mennyi idő telik el attól, hogy egy esemény megjelenik az MQTT brókeren, addig, amíg a böngészőben nyitott WebSocket-kapcsolaton megérkezik.

A második mutató méréséhez szintetikus, a Frigate üzenetformátumával megegyező eseményeket publikáltam a brókerre. Ez fontos módszertani pont: a mért érték kizárólag a rendszer saját átjátszási láncát – bróker, backend feldolgozás, WebSocket-továbbítás – jellemzi, és nem tartalmazza a Frigate detektálási idejét. Ez a szétválasztás szándékos: a detektálási időt a 3.6.3. szakasz méri külön, az átjátszási lánc pedig az, amiért a jelen munka felelős.

10. táblázat: Az állapotfigyelő hurok periódusa és az MQTT → WebSocket átjátszási késleltetés

Mutató	Érték
Az állapotfigyelő hurok átlagos periódusa (s)	5,380
A periódus szórása (s)	0,444

Átjátszási késleltetés — átlag (ms)	0,7
Átjátszási késleltetés — 95. percentilis (ms)	0,9
Illesztett eseménypárok száma	30
A mérés hossza (s)	120

Az eredmény a 10. táblázatban látható. Az átjátszási lánc átlagos késleltetése 0,7 ezredmásodperc, a 95. percentilis 0,9 ezredmásodperc, és a mérés alatt publikált mind a harminc esemény hiánytalanul megérkezett a WebSocket-kliensig. Ez a nagyságrend a felhasználói felületen érzékelhetetlen: az esemény gyakorlatilag abban a pillanatban jelenik meg a képernyőn, amikor a bróker megkapta. A mért érték visszaigazolja a 3.3. szakaszban leírt döntést, hogy a rendszer nem lekérdezéssel, hanem push-alapú közvetítéssel dolgozik – egy másodperces periódusú lekérdezés ugyanezt átlagosan félmásodpercre, tehát több mint hétszázszorosára növelte volna, ráadásul üresjárat terhelés árán.

Az állapotfigyelő hurok mért periódusa 5,38 másodperc, 0,444 másodperces szórással. Az öt másodperces névleges értéket tehát átlagosan 0,38 másodperccel lépi túl, ami pontosan a kamerák lekérdezési körének ideje: a hurok sorosan kérdezi le a node-okat, majd a teljes kör után vár öt másodpercet, így a tényleges periódus a névleges érték és a kör hosszának összege. A viszonylag nagy szórást az okozza, hogy a mérés idején az egyik node nem válaszolt, és annak lekérdezése minden körben a teljes kétmásodperces időkorlátig tartott, míg az elérhető node néhány ezredmásodperc alatt válaszolt. Sok kamera – vagy sok elérhetetlen node – esetén ez a hurok párhuzamosításával lenne csökkenthető; a jelenlegi kameraszámnál nem jelent szűk keresztmetszetet, de a rendszer egyik azonosított továbbfejlesztési pontja.

3.6.5. Az élőkép architektúrájának mérése

A negyedik mérés azt az architektúraváltást igazolja, amelyet a 3.5.3. szakasz ismertetett. Az összehasonlítás ugyanazon a rendszeren, ugyanazokkal a kamerákkal készült, egyetlen néző mellett.

11. táblázat: Az élőkép két megvalósításának összehasonlítása

Szempont	MJPEG-proxy (korábbi)	go2rtc MSE (jelenlegi)
Megjelenített felbontás	640×360 (a detekciós méret)	1280×720 (a telefon natív mérete)
Képfrissítés	3,5 kép/s (a detekciós képsebességhez kötve)	A telefon képsebessége, a detekciótól függetlenül
Frigate processzorterhelése nézőnként	+26 százalékpont	+2 százalékpont
Végponttól végpontig mért	Nem mérve; szemmel	0,03–0,07 s

késleltetés	láthatóan magasabb	
Szerveroldali újrakódolás	JPEG-kódolás nézőnként	Nincs, csak konténerformátum-váltás

A 11. táblázat legfontosabb sora a processzorterhelés: a korábbi megoldás minden egyes megnyitott kamerakártyáért egy teljes JPEG-kódolót futtatott, tehát négy egyszerre nézett kamera önmagában több mint egy processzormagnyi terhelést jelentett – ráadásul azért, hogy a felhasználó a rögzítettől rosszabb minőségű képet lásson. A jelenlegi megoldásban a szerver nem kódol, csak továbbít, így a nézők száma gyakorlatilag nem befolyásolja a szerver terhelését.

3.6.6. A képsebesség fényfüggése – a telefonoldali korlát

Az ötödik mérés a rendszer egyik legfontosabb, ugyanakkor a legkevésbé nyilvánvaló korlátját dokumentálja. Az élőkép képsebessége nem a szervertől, nem a hálózattól és nem a beállított értéktől függ, hanem a kamera előtti megvilágítástól.

Az Android kameraszolgáltatásának automatikus expozíciója gyenge fényben megnöveli az expozíciós időt, hogy a kép ne legyen alulexponált. A hosszabb expozíciós idő viszont fizikai korlátot szab a másodpercenként előállítható képkockák számának: ha egy képkocka rögzítése egy másodpercig tart, akkor a szenzor legfeljebb egy képet tud adni másodpercenként. Az alkalmazásban beállított képsebesség ilyenkor felső korlát, nem elérendő érték.

A jelenség egy node zseblámpájának ki- és bekapcsolásával reprodukálható. A rendszer a valós, mért képsebességet a kamerakártyán megjeleníti, így a hatás közvetlenül megfigyelhető: sötétben 3,3 kép/másodperc, a zseblámpa bekapcsolása után – a jelenetre változatlanul ugyanaz a kamera néz – 11,3 kép/másodperc, majd az expozíció beállása után 15 kép/másodperc fölé. Egy korábbi, sötétebb környezetben végzett mérésben ugyanez az ellentét 0,9 és 16,6 kép/másodperc között jelentkezett.



18. ábra: A képsebesség fényfüggése ugyanazon a node-on. A kártya bal alsó sarkában megjelenő, ténylegesen mért képsebesség sötétben 3,3 kép/s, a zseblámpa bekapcsolása után 11,3 kép/s – a jelenet és minden szerveroldali beállítás változatlan.

A 18. ábra ugyanazt a kamerát mutatja a zseblámpa kikapcsolt és bekapcsolt állapotában. A mérés következtetése egyértelmű: ez a korlát szerveroldalról nem kezelhető. Sem a detekciós képsebesség emelése, sem a hálózati beállítások változtatása nem befolyásolja, mert a képkockák egyszerűen nem jönnek létre. A gyakorlati megoldás a telepítés oldalán van: a megfigyelt terület megfelelő megvilágítása, vagy a fényforrásokat is figyelembe vevő kamerapozíció. Ez a felismerés a rendszer telepítési útmutatójába tartozik, nem a hibalistájába.

Ugyanez a hatás magyarázza, miért fontos, hogy a rendszer a **ténylegesen mért** képsebességet jelenítse meg a beállított érték helyett. A beállított érték a felhasználót félrevezetné: azt sugallná, hogy a rendszer huszonöt képet ad másodpercenként, miközben ténylegesen hármat.

3.6.7. Megfigyelt hibamódok

A folyamatos, éles üzemű tesztelés több olyan hibamódot is felszínre hozott, amelyek megléte a rendszer megbízhatóságának reális megítéléséhez tartozik.

A leggyakoribb hibamód a kamera-node váratlan elérhetetlensége. Ez a rendszer szempontjából nem hiba, hanem üzemszerű állapot: az állapotfigyelő hurok legkésőbb hét másodpercen belül észleli, a felület pedig egyértelműen jelzi, miközben a többi node zavartalanul működik tovább. A 3. ábra pontosan ezt az állapotot mutatja.

A második, alattomosabb hibamód a telefon videókódolójának elakadása. Ilyenkor a készülék hálózatilag elérhető marad – az állapotleíró válaszol, az akkumulátor- és Wi-Fi-adatok frissülnek, a node a felületen „online” állapotban van –, a H.264 végpont viszont üres választ ad, tehát videó nem érkezik. A jelenség a tesztelés során ténylegesen előfordult, és az MJPEG végpont eközben normálisan működött, ami megerősíti, hogy a hiba a hardveres kódoló munkamenetében keletkezik, nem a kamerában vagy a hálózatban. Az alkalmazás nem kínál olyan távoli vezérlést, amellyel a kódoló munkamenete újraindítható lenne; a felbontás és a képsebesség átállítása – tehát a kamera-munkamenet kényszerített újrakonfigurálása – sem oldotta fel az állapotot. Az egyetlen ismert megoldás az alkalmazás újraindítása a készüléken. A rendszernek ezért érdemes lenne külön jeleznie ezt az állapotot, mert jelenleg egy „online, de kép nélküli” node vizuálisan alig különböztethető meg a normál működéstől.

A harmadik hibamód a tűzdetektor téves riasztása gyenge fényben. A modell éjszakai, erősen zajos képeken ötven-hatvan százalék körüli magabiztossággal jelez füstöt ott, ahol nincs. A jelenség oka feltehetően az, hogy a nagy erősítéssel készült képen a szenzor zaj textúrája statisztikailag hasonlít arra, amit a modell füstként tanult meg. A jelenlegi ötven százalékos küszöb ezeket nem szűri ki. Kézenfekvő ellenintézkedés lenne a küszöb megemelése, illetve a képfényesség alapján egy éjszakai üzemmód bevezetése, amelyben a detektor magasabb küszöbbel dolgozik – ez a rendszer egyik konkrét továbbfejlesztési iránya.

3.7. A megoldás korlátai

A rendszer korlátainak összege számbavétele az értékelés része. Az alábbi korlátok mindegyike tudatosan vállalt kompromisszum vagy azonosított hiányosság, nem pedig felderítetlen hiba.

A biztonsági modell a megbízható helyi hálózat feltételezésére épül – a 3.5.9. szakaszban felsorolt döntések mindegyike ebből ered, és a rendszer közvetlen internetes kitettsége mindegyiket sebezhetőséggé változtatná.

A rendszer állapota a memóriában él. A kameralista minden induláskor a Frigate konfigurációjából épül újra, ami egyetlen csomópontra és kis kameraszámra megfelelő, de kizárja a többcsomópontos üzemet és a hosszabb távú állapot történet vezetését.

A kamera-node megbízhatósága a telefontól függ. Az Android energiagazdálkodása leállíthatja a háttérben futó alkalmazást, a videókódoló elakadhat, és egyik esetre sincs távoli helyreállítási mód. Egy folyamatosan üzemelő telefon hőterhelése hosszú távon gyorsítja az akkumulátor degradációját, a készülék pedig védtelen kültéri környezetbe nem helyezhető ki.

A protokollcserével néhány telemetriai adat elveszett, ezek közül a legérzékenyebb a töltési állapot. Egy folyamatos üzemre szánt node tápellátásának megszűnése jelenleg csak közvetetten, az akkumulátorszint csökkenéséből ismerhető fel, nem pedig azonnal, a töltő lecsatlakozásának pillanatában.

A tűzdetekciós modell nem saját betanítású, hanem nyilvánosan elérhető súlykészlet. Ennek megfelelően a viselkedése nem a jelen rendszer telepítési körülményeire van optimalizálva – ez okozza a gyenge fényben tapasztalt téves riasztásokat. A modell saját gyűjtésű mintákon való finomhangolása kézenfekvő javítási irány.

Végül a rendszer jelenleg a telefon szenzorai közül gyakorlatilag csak a kamerát és az akkumulátor-, illetve Wi-Fi-állapotot használja. A mikrofon, a gyorsulásmérő és a giroszkóp mint eseményforrás nincs bevonva, holott az architektúra – az MQTT-alapú, szétcsatolt eseményközvetítés miatt – ezek fogadására különösebb átalakítás nélkül alkalmas lenne.

4. Összegzés

A szakdolgozat célja egy felhőfüggetlen, lokális hálózaton működő, mesterséges intelligenciával támogatott kamerarendszer (SwarmCam) tervezése és megvalósítása volt, amely feleslegessé vált Android okostelefonokat integrál dedikált kamerahardver nélkül. A megvalósítás igazolta, hogy egy hálózati felderítésre, konténerizált mikroszolgáltatásokra (Frigate, saját FastAPI backend, önálló YOLO-alapú tűzdetektor) és eseményvezérelt kommunikációra (MQTT, WebSocket) épülő architektúra alkalmas ilyen heterogén, régi eszközökből álló megfigyelőhálózat kiszolgálására, gyakorlatilag zéró manuális konfigurációval az egyes kamerák felvételekor.

A kitűzött részcélok – az automatikus felderítés, a felhőfüggetlen szerverkörnyezet, az AI-alapú detekció (objektum-, valamint tűz-/füstfelismerés, kiegészítve arcfelismeréssel), a valós idejű eseménykezelés és a kezelőfelület – megvalósultak. A rendszer moduláris felépítése (különálló Docker konténerek, jól definiált MQTT/WebSocket interfészek) lehetővé teszi az egyes komponensek önálló továbbfejlesztését anélkül, hogy a többi részt érintené.

A dolgozatban bemutatott rendszer saját hozzájárulást jelentő komponenseit – a hálózati felderítést és az ahhoz tartozó ujjlenyomat-vizsgálatot, a FastAPI backendet és annak valós idejű kommunikációs rétegét (állapotfigyelő hurok, MQTT-átjátszás, WebSocket-elosztás), az élőkép-architektúrát a go2rtc MSE-relayjével, a kameraforgatás feldolgozási láncát, a tűz-/füstdetekciós mikroszolgáltatás teljes Python-csővezetékét, valamint a React alapú, asztali és mobil elrendezést egyaránt kiszolgáló kezelőfelületet – a szerző önállóan tervezte és valósította meg; ide tartoznak a 3. fejezetben indokolt tervezési döntések és a 3.6. szakaszban közölt mérési módszertan is. A rendszer ugyanakkor több készen kapott, harmadik féltől származó komponensre épül: a Frigate NVR-re és a benne futó go2rtc-re, az Android IP Camera alkalmazásra, a Mosquitto üzenetbrókerre, valamint a tűz-/füstdetekcióban használt, előre betanított YOLO-modellre (yolov26-fire-detection [10]) – ez utóbbit a szerző nem tanította be, csupán a saját csővezetékébe integrálta.

4.1. Jövőbeli fejlesztési lehetőségek

A jelenlegi, működő rendszer több irányban is bővíthető:

- ONVIF-kompatibilis, dedikált IP kamerák (pl. PTZ funkcióval) bevonása a hálózatba, a discovery modul kiterjesztésével.
- A tűz-/füstdetekciós modell további finomhangolása (transfer learning) saját gyűjtésű adathalmazon, a téves riasztások arányának csökkentésére.
- Push értesítést támogató natív mobilalkalmazás fejlesztése a webes dashboard mellé, amely háttérben futva is riasztást tud küldeni.

- A Google Coral TPU-alapú gyorsítás kiértékelése alternatívaként az NVIDIA GPU mellett, alacsonyabb energiafogyasztású telepítésekhez.
- A telefonok további beépített szenzorainak (mikrofon, gyorsulásmérő, giroszkóp) bevonása újabb eseményforrásként, a jelenleg is felhasznált kameraképet és akkumulátor-/töltésállapotot kiegészítve.
- A kamera-node videó kódolójának elakadását felismerő figyelőmechanizmus: a rendszer jelenleg „online” állapotúnak mutatja azt a node-ot is, amely hálózatilag válaszol, de videót már nem szolgáltat.
- A tűz-/füstdetektor fényviszony-érzékeny küszöbölése, hogy az éjszakai, erősen zajos képeken tapasztalt téves füstriasztások megszűnjenek.

Irodalom

- [1] Frigate NVR – Documentation. [Online]. Elérhető: <https://docs.frigate.video/>
- [2] OASIS, MQTT Version 5.0 – OASIS Standard. [Online]. Elérhető: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [3] ONVIF, ONVIF Core Specification. [Online]. Elérhető: <https://www.onvif.org/specs/>
- [4] P. Khlebovich, IP Webcam. [Mobilalkalmazás]. Google Play Áruház.
- [5] I. Goodfellow, Y. Bengio, A. Courville, Deep Learning. Cambridge, MA: MIT Press, 2016.
- [6] Ultralytics, YOLO Docs. [Online]. Elérhető: <https://docs.ultralytics.com/>
- [7] V. Forti, C. P. Baldé, R. Kuehr, G. Bel, The Global E-waste Monitor 2020. Bonn/Geneva/Rotterdam: United Nations University/UNITAR, ITU, ISWA, 2020.
- [8] FastAPI – Documentation. [Online]. Elérhető: <https://fastapi.tiangolo.com/>
- [9] Docker, Docker Compose – Documentation. [Online]. Elérhető: <https://docs.docker.com/compose/>
- [10] S. Al-Haismawi, yolov26-fire-detection [Betanított modell]. Hugging Face. [Online]. Elérhető: <https://huggingface.co/SalahALHaismawi/yolov26-fire-detection>
- [11] React – Documentation. [Online]. Elérhető: <https://react.dev/>
- [12] DigitallyRefined, Android IP Camera. [Mobilalkalmazás]. F-Droid. [Online]. Elérhető: <https://github.com/DigitallyRefined/android-ip-camera>
- [13] Microsoft, ONNX Runtime – Documentation. [Online]. Elérhető: <https://onnxruntime.ai/docs/>
- [14] A. Ivanov, go2rtc – Ultimate camera streaming application. [Online]. Elérhető: <https://github.com/AlexxIT/go2rtc>
- [15] MDN Web Docs, Media Source Extensions API. [Online]. Elérhető: https://developer.mozilla.org/en-US/docs/Web/API/Media_Source_Extensions_API

5. Mellékletek

A mellékletek a dolgozat fő fejezeteit feleslegesen terhelő kiegészítő anyagokat tartalmazzák.

5.1. Nyomtatott mellékletek

Jelen szakdolgozathoz különálló nyomtatott melléklet nem tartozik; az ábrák és táblázatok a dolgozat szövegében szerepelnek.

5.2. Elektronikus mellékletek

DVD vagy pendrive, amely tartalmazza a teljes szakdolgozatot *.docx és *.pdf formátumban, a védés során bemutatott prezentációt (*.pptx), valamint a SwarmCam rendszer forráskódját.